

What Are Numerical Methods?

Numerical Methods are techniques used to approximate solutions for mathematical problems that cannot be solved analytically. These methods use algorithms and computational techniques to obtain numerical solutions

• Examples:

- Solving equations
- Numerical integration
- Interpolation
- Differential equations

Why Use Numerical Methods?

- Many real-world problems cannot be solved exactly.
- Analytical solutions may not exist or may be too complex.
- Computers can efficiently handle large numerical computations.

A Brief History

- Ancient Times: Use of approximation in Babylonian & Egyptian mathematics.
- 17th Century: Newton's Method (Isaac Newton)
- 19th Century: Development of numerical integration & difference methods.
- 20th Century: Rise of digital computers boosted the development of complex algorithms

Common Numerical Methods

- Root-Finding: Bisection Method, Newton-Raphson Method
- Numerical Integration: Trapezoidal Rule, Simpson's Rule
- Differential Equations: Euler's Method, Runge-Kutta Methods
- Linear Systems: Gauss Elimination, LU Decomposition

Applications of Numerical Methods

- Engineering: Structural analysis, fluid dynamics, heat transfer
- Physics & Chemistry: Quantum mechanics, chemical reaction modeling
- Computer Science: Graphics, simulations, machine learning
- Finance: Risk analysis, pricing derivatives
- Environmental Science: Climate modeling, population dynamics

Real-World Examples

- Designing bridges using finite element methods
- Weather forecasting using numerical weather models
- Simulating airflow over aircraft wings
- Medical imaging (e.g., MRI, CT reconstruction)

Cramer's Rule (An Introduction to Solving Systems of Linear Equation)

Cramer's Rule is a mathematical theorem used to solve systems of linear equations with as many equations as unknowns, using determinants.

Conditions for Applying Cramer's Rule:

- The system must be a square system (same number of equations and unknowns).
- The determinant of the coefficient matrix must be non-zero.
- Only applicable to linear equations.

Why are they called linear equations? Because each equation has:

- Variables raised only to the power of 1.
- No multiplication between variables (e.g., no xy , x^2 , etc.).
- Graphically, each equation (in 3D) represents a plane, and the solution is the point where all three planes intersect.

Applications of Cramer's Rule:

- Used in solving small systems of linear equations
- Useful in theoretical mathematics and engineering
- Basis for understanding matrix-based solutions in linear algebra.

Problem Statement. Use Cramer's rule to solve

$$\begin{aligned}0.3x_1 + 0.52x_2 + x_3 &= -0.01 \\0.5x_1 + x_2 + 1.9x_3 &= 0.67 \\0.1x_1 + 0.3x_2 + 0.5x_3 &= -0.44\end{aligned}$$

Solution. The determinant D can be written as [Eq. (9.2)]

$$D = \begin{vmatrix} 0.3 & 0.52 & 1 \\ 0.5 & 1 & 1.9 \\ 0.1 & 0.3 & 0.5 \end{vmatrix}$$

The minors are [Eq. (9.3)]

$$A_1 = \begin{vmatrix} 1 & 1.9 \\ 0.3 & 0.5 \end{vmatrix} = 1(0.5) - 1.9(0.3) = -0.07$$

$$A_2 = \begin{vmatrix} 0.5 & 1.9 \\ 0.1 & 0.5 \end{vmatrix} = 0.5(0.5) - 1.9(0.1) = 0.06$$

$$A_3 = \begin{vmatrix} 0.5 & 1 \\ 0.1 & 0.3 \end{vmatrix} = 0.5(0.3) - 1(0.1) = 0.05$$

These can be used to evaluate the determinant, as in [Eq. (9.4)]

$$D = 0.3(-0.07) - 0.52(0.06) + 1(0.05) = -0.0022$$

Applying Eq. (9.5), the solution is

$$x_1 = \frac{\begin{vmatrix} -0.01 & 0.52 & 1 \\ 0.67 & 1 & 1.9 \\ -0.44 & 0.3 & 0.5 \end{vmatrix}}{-0.0022} = \frac{0.03278}{-0.0022} = -14.9$$

$$x_2 = \frac{\begin{vmatrix} 0.3 & -0.01 & 1 \\ 0.5 & 0.67 & 1.9 \\ 0.1 & -0.44 & 0.5 \end{vmatrix}}{-0.0022} = \frac{0.0649}{-0.0022} = -29.5$$

$$x_3 = \frac{\begin{vmatrix} 0.3 & 0.52 & -0.01 \\ 0.5 & 1 & 0.67 \\ 0.1 & 0.3 & -0.44 \end{vmatrix}}{-0.0022} = \frac{-0.04356}{-0.0022} = 19.8$$

Limitations of Cramer's Rule: For more than three equations, Cramer's rule becomes impractical because, as the number of equations increases, the determinants are time consuming to evaluate by hand (or by computer).

- Slow for large systems – Inefficient due to determinant calculations.
- Only for square matrices – Number of equations must equal number of unknowns.

- Fails if determinant is zero – Cannot solve if matrix is singular.
- Not practical for computation – Rarely used in real-world applications.

Iteration Method (Fixed-Point Iteration)

- A numerical method to solve equations of the form $f(x) = 0$
- Rewritten as $x = g(x)$, then:

$$x_{n+1} = g(x_n)$$
- Continue until $|x_{n+1} - x_n| < \epsilon$ (tolerance)
- Used when analytical methods are difficult or impossible

Solve $X = 0.5X^2 + 0.25$ by Iterative Method

We are solving the equation: $X = 0.5X^2 + 0.25$

Using Fixed-Point Iteration:

$$X_{n+1} = 0.5X_n^2 + 0.25$$

We try different initial guesses: 2, 1, and 0.1.

Initial Guess: $X_0 = 2$

$$X_1 = 0.5(2)^2 + 0.25 = 2.25$$

$$X_2 = 0.5(2.25)^2 + 0.25 = 2.78125$$

$$X_3 = 0.5(2.78125)^2 + 0.25 = 4.1172$$

$$X_4 = 8.729$$

$$X_5 = 38.349$$

Result: Diverging, does not converge.

Initial Guess: $X_0 = 1$

$$X_1 = 0.75$$

$$X_2 = 0.53125$$

$$X_3 = 0.391$$

$$X_4 = 0.3265$$

$$X_5 = 0.3033$$

$$X_6 = 0.296$$

$$X_7 = 0.2938$$

Result: Converges to ≈ 0.293 (Answer)

Initial Guess: $X_0 = 1$

$$X_1 = 0.75$$

$$X_2 = 0.53125$$

$$X_3 = 0.391$$

$$X_4 = 0.3265$$

$$X_5 = 0.3033$$

$$X_6 = 0.296$$

$$X_7 = 0.2938$$

Result: Converges to ≈ 0.293 (Answer)

The iterative method depends heavily on the initial guess. For $X_0 = 2$, the iteration diverges. For $X_0 = 1$ and $X_0 = 0.1$, the iteration converges. Approximate root: $X \approx 0.293$

>In numerical methods, convergence refers to the property of an iterative method where the sequence of approximations gets closer and closer to the actual solution as the number of iterations increases.

>Divergence refers to the situation where the sequence of approximations produced by an iterative method does not approach the exact solution and instead moves away from the true solution as iterations continue.

Solve $x^3 + x - 1 = 0$ (try yourself)

$x^2 - x - 1 = 0$	
$x^2 = x + 1$	$x^2 - x = 1$
$x = 1 + \frac{1}{x}$	$x(x - 1) = 1$
$x_{n+1} = 1 + \frac{1}{x_n}$	$x = \frac{1}{x - 1}$
Pick $x_1 = 2$	$x_{n+1} = \frac{1}{x_n - 1}$
$x_2 = 1 + \frac{1}{2} = 1.5$	Pick $x_1 = 1.6$
$x_3 = 1 + \frac{1}{1.5} = 1.6666$	$x_2 = \frac{1}{1.6 - 1} = 1.6666$
$x_4 = 1 + \frac{1}{1.6666} = 1.6$	$x_3 = \frac{1}{1.6666 - 1} = 1.5$
$x_5 = 1 + \frac{1}{1.6} = 1.625$	$x_4 = \frac{1}{1.5 - 1} = 2$
$x_6 = 1 + \frac{1}{1.625} = 1.612538462$	$x_5 = \frac{1}{2 - 1} = 1$
Converging to 1.618 ...	Not converging

Fixed-Point Iteration is a numerical method for solving equations of the form $X = f(X)$.

It is used in various fields where equations **cannot be solved analytically and iterative solutions are required.**

The method starts with an **initial guess and generates a sequence converging to the solution.**

Problem Statement. Use simple fixed-point iteration to locate the root of $f(x) = e^{-x} - x$.

Solution. The function can be separated directly and expressed in the form of Eq. (6.2) as

$$x_{i+1} = e^{-x_i}$$

Starting with an initial guess of $x_0 = 0$, this iterative equation can be applied to compute

i	x_i	ϵ_a (%)	ϵ_t (%)
0	0		100.0
1	1.000000	100.0	76.3
2	0.367879	171.8	35.1
3	0.692201	46.9	22.1
4	0.500473	38.3	11.8
5	0.606244	17.4	6.89
6	0.545396	11.2	3.83
7	0.579612	5.90	2.20
8	0.560115	3.48	1.24
9	0.571143	1.93	0.705
10	0.564879	1.11	0.399

Thus, each iteration brings the estimate closer to the true value of the root: 0.56714329.

Real-World Applications of Fixed-Point Iteration Method

1) Heat Transfer Engineering: Used to solve steady-state heat conduction problems.

Example: Solving nonlinear temperature equations in furnace walls.

2. Electrical Power Flow Analysis

- Common in load flow studies in power grids.
- Solves nonlinear algebraic equations using iterative methods.
- Example: Solving $V = f(V)$ in voltage distribution systems.

3. Structural Engineering

- Used in analyzing nonlinear stress-strain behavior.
- Materials like concrete and plastic exhibit nonlinearity.
- Iteration helps compute deflections and stress under loads.

4. Computational Fluid Dynamics (CFD)

- Iteratively solve velocity, pressure, and flow equations.
- Widely used in aerodynamics, weather models, and pipe flow systems.
- Update rule: $U^{n+1} = f(U^n)$

5. Economics and Equilibrium Modeling

- Used in supply-demand equilibrium models.
- Solves utility or production functions iteratively.
- Example: $x = f(x)$, where f represents market adjustment function.

6. Robotics and Control Systems

- Applied in inverse kinematics and PID controller tuning.
- Iterative refinement of robot joint angles or system gains.
- Embedded systems use fixed-point due to computational efficiency.

Convergence Condition

The iteration will converge to a solution if: $|g'(x)| < 1$ near the root

Otherwise, the method may diverge

Always check derivative before using

Solving Nonlinear Temperature Equations in Furnace Walls

- We consider a nonlinear equation often arising in heat transfer problems:

$$T = \alpha T^2 + \beta$$

- This equation models temperature distribution with nonlinear effects such as radiation or variable conductivity.

Rearranged Form for Iteration

- Rearranged as: $T_{i+1} = \alpha T_i^2 + \beta$

- This allows us to use fixed-point iteration to approximate the solution.

1. Choose initial guess T_0
2. Compute $T_1 = \alpha T_0^2 + \beta$
3. Repeat: $T_{n+1} = \alpha T_n^2 + \beta$
4. Stop when $|T_{n+1} - T_n| < \epsilon$ (convergence criteria)

Convergence Criteria

- For convergence: $|f'(T)| = |2\alpha T| < 1$

So, convergence depends on: Value of α , Initial guess T_0 , Tolerance ϵ

Given: $\alpha = 0.01$, $\beta = 200$, $T_0 = 300$

$$\rightarrow T_1 = \alpha(300)^2 + 200 = 1100$$

$$\rightarrow T_2 = \alpha(1100)^2 + 200 = 12300$$

$\rightarrow$ Divergence occurs! Better α or method needed.

Bracketing Method (Root-Finding)

- Numerical technique for finding roots of equations.
- Based on Intermediate Value Theorem: If $f(a) \cdot f(b) < 0$, a root exists in $[a, b]$
- Requires a function to be continuous in the interval.

Types of Bracketing Methods

1. Bisection Method: Splits interval and chooses half with sign change.
2. False Position (Regula Falsi): Uses secant line between $f(a)$ and $f(b)$ to estimate root.
3. Brent's Method: Combines bisection, secant, and interpolation for faster convergence.

Steps of Bracketing Method

1. Choose a and b such that $f(a) \cdot f(b) < 0$
2. Compute root approximation (midpoint or secant line)
3. Check for convergence ($|f(x)| < \text{tolerance}$)
4. Narrow the interval and repeat

Example: $f(x) = x^3 - 4x + 1$

Try interval $a = 0$, $b = 1$

$f(0) = 1$, $f(1) = -2 \rightarrow$ Signs opposite. Root lies between $[0, 1]$

Apply bisection or false position method

Bisection Method

Step 1: Choose lower x_l and upper x_u guesses for the root such that the function changes sign over the interval. This can be checked by ensuring that $f(x_l)f(x_u) < 0$.

Step 2: An estimate of the root x_r is determined by

$$x_r = \frac{x_l + x_u}{2}$$

Step 3: Make the following evaluations to determine in which subinterval the root lies:

(a) If $f(x_l)f(x_r) < 0$, the root lies in the lower subinterval. Therefore, set $x_u = x_r$ and return to step 2.

(b) If $f(x_l)f(x_r) > 0$, the root lies in the upper subinterval. Therefore, set $x_l = x_r$ and return to step 2.

(c) If $f(x_l)f(x_r) = 0$, the root equals x_r ; terminate the computation.

Problem: Example-5.3

Determine the drag coefficient c needed for a parachutist of mass $m = 68.1$ kg to have a velocity of 40 m/s after freefalling for time $t = 10$ s. Note: The acceleration due to gravity is 9.81 m/s².

The solution to the dilemma is provided by numerical methods for roots of equations. To solve the problem using numerical methods, it is conventional to reexpress Eq. (PT2.3). This is done by subtracting the dependent variable v from both sides of the equation to give

$$f(c) = \frac{gm}{c} (1 - e^{-(c/m)t}) - v \quad (\text{PT2.4})$$

The value of c that makes $f(c) = 0$ is, therefore, the root of the equation. This value also represents the drag coefficient that solves the design problem.

$$f(c) = \frac{9.81(68.1)}{c} (1 - e^{-(c/68.1)10}) - 40$$

Solution. The first step in bisection is to guess two values of the unknown (in the present problem, c) that give values for $f(c)$ with different signs. From Fig. 5.1, we can see that the function changes sign between values of 12 and 16. Therefore, the initial estimate of the root x_r lies at the midpoint of the interval

$$x_r = \frac{12 + 16}{2} = 14$$

This estimate represents a true percent relative error of $\varepsilon_t = 5.3\%$ (note that the true value of the root is 14.8011). Next we compute the product of the function value at the lower bound and at the midpoint:

$$f(12)f(14) = 6.114(1.611) = 9.850$$

which is greater than zero, and hence no sign change occurs between the lower bound and the midpoint. Consequently, the root must be located between 14 and 16. Therefore, we create a new interval by redefining the lower bound as 14 and determining a revised root estimate as

$$x_r = \frac{14 + 16}{2} = 15$$

which represents a true percent error of $\varepsilon_t = 1.3\%$. The process can be repeated to obtain refined estimates. For example,

$$f(14)f(15) = 1.611(-0.384) = -0.619$$

Therefore, the root is between 14 and 15. The upper bound is redefined as 15, and the root estimate for the third iteration is calculated as

$$x_r = \frac{14 + 15}{2} = 14.5$$

which represents a percent relative error of $\varepsilon_t = 2.0\%$. The method can be repeated until the result is accurate enough to satisfy your needs.

Problem Statement. Continue Example 5.3 until the approximate error falls below a stopping criterion of $\varepsilon_s = 0.5\%$. Use Eq. (5.2) to compute the errors.

Solution. The results of the first two iterations for Example 5.3 were 14 and 15. Substituting these values into Eq. (5.2) yields

$$|\varepsilon_a| = \left| \frac{15 - 14}{15} \right| 100\% = 6.667\%$$

Recall that the true percent relative error for the root estimate of 15 was 1.3%. Therefore, ε_a is greater than ε_t . This behavior is manifested for the other iterations:

Iteration	x_l	x_u	x_r	ε_a (%)	ε_t (%)
1	12	16	14		5.413
2	14	16	15	6.667	1.344
3	14	15	14.5	3.448	2.035
4	14.5	15	14.75	1.695	0.345
5	14.75	15	14.875	0.840	0.499
6	14.75	14.875	14.8125	0.422	0.077

Thus, after six iterations ε_a finally falls below $\varepsilon_s = 0.5\%$, and the computation can be terminated.

Are iteration and bisection method for same problem ok?

Yes, iteration method and bisection method can absolutely be used on the same problem, especially when the problem is to find a root of a nonlinear equation, like: $f(x)=0$

- **Both Bisection and Iteration methods are used to find roots of nonlinear equations.**
- **We compare their approach, convergence, and use on the same problem.**
- **Example problem: Solve $f(x) = x - \cos(x) = 0$**

Bisection Method Overview	Iteration Method Overview
Type: Bracketing method	Type: Open method (Fixed-point)
Requires interval $[a, b]$ with $f(a)*f(b) < 0$	Rewrite $f(x) = 0$ as $x = g(x)$
Guaranteed convergence (slow)	Converges if $ g'(x) < 1$ near the root
Divides interval in half at each step	Faster than bisection (if convergent)

Solving $x = \cos(x)$

Bisection Method:

$$f(x) = x - \cos(x)$$

Choose $[0, 1]$ where $f(0)*f(1) < 0$

Iteration Method:

$$\text{Use } g(x) = \cos(x)$$

$$x_0 = 0.5 \rightarrow x_1 = \cos(0.5) \rightarrow x_2 = \cos(x_1) \dots$$

Comparison: Bisection vs Iteration

Bisection Method:

- - Always converges
- - Slower (linear convergence)
- - Needs interval with sign change

Iteration Method:

- - May not converge
- - Faster (if converges)
- - Needs good initial guess

- Both methods solve $f(x) = 0$
- Bisection: Safe and slow
- Iteration: Fast but sensitive
- Choose based on problem nature and available data

False Position Method (Regula Falsi)

Concept of False Position Method

- Bracketing method for root finding.
- Uses linear interpolation between two points.
- Requires function to be continuous in $[a, b]$ and $f(a) \cdot f(b) < 0$.
- More efficient than Bisection in some cases.

Formula

$$x_r = b - \frac{f(b)(a - b)}{f(a) - f(b)}$$

False Position Method Algorithm

Input:

Where:

- x_r = estimated root
- a, b = current interval
- $f(a), f(b)$ = function values at the endpoints
- Function $f(x)$
- Initial guesses a and b such that $f(a) \cdot f(b) < 0$
- Tolerance ε
- Maximum number of iterations N

Steps

1. Check initial values:

- If $f(a) \cdot f(b) \geq 0$, then the method fails (no sign change).

2. Start loop (for $i = 1$ to N):

- Calculate root estimate using the formula:

$$x_r = b - \frac{f(b)(a - b)}{f(a) - f(b)}$$

3. Evaluate $f(x_r)$:

- If $|f(x_r)| < \varepsilon$, then stop and return x_r as root.

4. Update interval:

- If $f(a) \cdot f(x_r) < 0$, then set $b = x_r$
- Else set $a = x_r$

5. Repeat until tolerance is met or max iterations reached.

False Position Method Algorithm

Pseudocode:

```
python

Input: f(x), a, b, ε, N
If f(a) * f(b) ≥ 0:
    Print "Invalid interval"
    Exit

For i = 1 to N:
    xr = b - (f(b) * (a - b)) / (f(a) - f(b))

    If |f(xr)| < ε:
        Return xr

    If f(a) * f(xr) < 0:
        b = xr
    Else:
        a = xr

Return xr
```

Real Example: $f(x) = x^3 - 4x + 1$

Initial Interval: [0, 1] $f(0) = 1, f(1) = -2 \rightarrow$ Sign change exists

Step 1: $x_r = 1 - [-2 * (0-1)] / [1 - (-2)] = 0.3333$ $f(0.3333) \approx -0.2962$

Step 2: New interval: [0, 0.3333] $\rightarrow$ Repeat steps

Iteration	a	b	f(a)	f(b)	x_r	$f(x_r)$
0	0.0000	1.0000	1.0000	-2.0000	0.3333	-0.2962
1	0.0000	0.3333	1.0000	-0.2962	0.2573	-0.0120
2	0.0000	0.2573	1.0000	-0.0120	0.2542	-0.0002
3	0.0000	0.2542	1.0000	-0.0002	0.2541	≈ 0.0000

After 4 iterations, the approximate root is: 0.2541

Advantages:

- Faster than bisection in many cases
- Bracketed method ensures root lies in interval

Disadvantages:

- May converge slowly if one function value is small
- Not always faster than bisection

Bracketing Methods

- $f(x)$ has to be continuous in the interval $[x_l, x_u]$ and $f(x_l)f(x_u) < 0$
- Always converge
- Usually slower than open methods

Bisection Method

- Slow but guarantee the best **worst-case** convergent rate.

False-position method

- In general performs better than bisection method (with some exceptions).

False Position vs Bisection (regular case)

Solution. The results of the first two iterations for Example 5.3 were 14 and 15. Substituting these values into Eq. (5.2) yields

$$|\varepsilon_a| = \left| \frac{15 - 14}{15} \right| 100\% = 6.667\%$$

Recall that the true percent relative error for the root estimate of 15 was 1.3%. Therefore, ε_a is greater than ε_t . This behavior is manifested for the other iterations:

Iteration	x_l	x_u	x_r	ε_a (%)	ε_t (%)
1	12	16	14		5.413
2	14	16	15	6.667	1.344
3	14	15	14.5	3.448	2.035
4	14.5	15	14.75	1.695	0.345
5	14.75	15	14.875	0.840	0.499
6	14.75	14.875	14.8125	0.422	0.077

Thus, after six iterations ε_a finally falls below $\varepsilon_s = 0.5\%$, and the computation can be terminated.

Problem Statement. Use the false-position method to determine the root of the same equation investigated in Example 5.1 [Eq. (E5.1.1)].

Solution. As in Example 5.3, initiate the computation with guesses of $x_l = 12$ and $x_u = 16$.

First iteration:

$$\begin{aligned}x_l &= 12 & f(x_l) &= 6.1139 \\x_u &= 16 & f(x_u) &= -2.2303 \\x_r &= 16 - \frac{-2.2303(12 - 16)}{6.1139 - (-2.2303)} = 14.309\end{aligned}$$

which has a true relative error of 0.88 percent.

Second iteration:

$$f(x_l)f(x_r) = -1.5376$$

Therefore, the root lies in the first subinterval, and x_r becomes the upper limit for the next iteration, $x_u = 14.9113$:

$$\begin{aligned}x_l &= 12 & f(x_l) &= 6.1139 \\x_u &= 14.9309 & f(x_u) &= -0.2515 \\x_r &= 14.9309 - \frac{-0.2515(12 - 14.9309)}{6.1139 - (-0.2515)} = 14.8151\end{aligned}$$

which has true and approximate relative errors of 0.09 and 0.78 percent. Additional iterations can be performed to refine the estimate of the roots.

A Case Where Bisection Is Preferable to False Position

Problem Statement. Use bisection and false position to locate the root of

$$f(x) = x^{10} - 1$$

between $x = 0$ and 1.3 .

Solution. Using bisection, the results can be summarized as

Iteration	x_l	x_u	x_r	ϵ_a (%)	ϵ_t (%)
1	0	1.3	0.65	100.0	35
2	0.65	1.3	0.975	33.3	2.5
3	0.975	1.3	1.1375	14.3	13.8
4	0.975	1.1375	1.05625	7.7	5.6
5	0.975	1.05625	1.015625	4.0	1.6

Thus, after five iterations, the true error is reduced to less than 2 percent. For false position, a very different outcome is obtained:

Iteration	x_l	x_u	x_r	ϵ_a (%)	ϵ_t (%)
1	0	1.3	0.09430		90.6
2	0.09430	1.3	0.18176	48.1	81.8
3	0.18176	1.3	0.26287	30.9	73.7
4	0.26287	1.3	0.33811	22.3	66.2
5	0.33811	1.3	0.40788	17.1	59.2

Real-Life Incidence Example: Estimating Pipe Flow Rate (Engineering)

Scenario:

A civil engineer is designing a water supply pipeline. To ensure proper flow, they must solve **Darcy-Weisbach equation**, which is **non-linear** in terms of the flow velocity.

Let's simplify to a case where the engineer needs to solve:

$$f(v) = v^3 - 2v - 5 = 0$$

to find the velocity v of water that satisfies certain constraints.

Applying False Position Method

Step 1: Choose interval $[a, b]$ such that $f(a) \cdot f(b) < 0$

Let:

- $a = 2$, then $f(2) = 8 - 4 - 5 = -1$
- $b = 3$, then $f(3) = 27 - 6 - 5 = 16$

So, $f(a) \cdot f(b) = (-1)(16) < 0 \rightarrow$ valid.

Step 2: Formula

$$c = b - \frac{f(b)(a - b)}{f(a) - f(b)}$$

Iteration 1:

$$c_1 = 3 - \frac{f(3)(2-3)}{f(2)-f(3)} = 3 - \frac{16 \cdot (-1)}{-1-16} = 3 - \frac{-16}{-17} = 3 - 0.9412 = 2.0588$$

$$f(2.0588) \approx (2.0588)^3 - 2(2.0588) - 5 \approx 8.722 - 4.1176 - 5 \approx -0.3956$$

Now:

- $f(2) = -1$
- $f(2.0588) = -0.3956$

So $f(c) \cdot f(a) > 0$, update $a = c$, keep $b = 3$

Iteration	a	b	$f(a)$	$f(b)$	v_r	$f(v_r)$
0	2.0000	3.0000	-1.0000	16.0000	2.0588	-0.3945
1	2.0588	3.0000	-0.3945	16.0000	2.0753	-0.1575
2	2.0753	3.0000	-0.1575	16.0000	2.0807	-0.0620
3	2.0807	3.0000	-0.0620	16.0000	2.0827	-0.0243
4	2.0827	3.0000	-0.0243	16.0000	2.0834	-0.0095
5	2.0834	3.0000	-0.0095	16.0000	2.0837	-0.0037
6	2.0837	3.0000	-0.0037	16.0000	2.0838	-0.0014
7	2.0838	3.0000	-0.0014	16.0000	2.0839	-0.0005
8	2.0839	3.0000	-0.0005	16.0000	2.0839	-0.0002

Wrong Calculation, so solve yourself!

Interpretation in Real-Life:

This velocity root helps the engineer **design pipe diameter**, **select pump power**, and **ensure pressure safety** without needing complex algebraic solutions. It's especially useful when dealing with real-world **nonlinear equations** in:

- Civil Engineering (Fluid Mechanics)
- Electrical Engineering (nonlinear resistive circuits)
- Environmental Science (pollutant transport equations)

Advantages of False Position method

Faster than Bisection method

Always converges ($f(a) \cdot f(b) < 0$)

No derivative needed (unlike Newton-Raphson)

Newton-Raphson Method: A Numerical Root-Finding Algorithm

The **Newton-Raphson method** is a powerful **iterative technique** used to find **roots of nonlinear equations** of the form:

$$f(x) = 0$$

Formula:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

Where:

- x_n = current approximation
- x_{n+1} = next approximation
- $f(x)$ = function
- $f'(x)$ = derivative of the function

Algorithm Steps:

1. Choose an initial guess x_0
2. Compute next value using:

$$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)}$$

3. Repeat the formula until the solution converges:

$$|x_{n+1} - x_n| < \epsilon \text{ (small tolerance)}$$

Example: Solve $x^2 - 2 = 0$

Goal: Find $\sqrt{2}$

- $f(x) = x^2 - 2$
- $f'(x) = 2x$

Let's start with $x_0 = 1$

Iteration	x_n	$f(x_n)$	$f'(x_n)$	x_{n+1}
0	1.0000	-1.0000	2.0000	$1 + 0.5 = 1.5$
1	1.5	0.25	3.0	$1.5 - 0.0833 = 1.4167$
2	1.4167	0.0069	2.833	≈ 1.4142

 Converges to $\sqrt{2} \approx 1.4142$

Advantages

Fast convergence (quadratic)

Simple to implement

Great for well-behaved functions

Disadvantages

Needs derivative $f'(x)$

Can diverge if initial guess is poor

Fails at points where $f'(x) = 0$

- The Newton-Raphson method is known for its fast (quadratic) convergence, which means it gets very close to the true solution very quickly, especially near the root.

Comparison with Other Methods:

Method	Convergence Rate	Speed
Bisection	Linear	Slow
Secant	Superlinear	Moderate
Newton-Raphson	Quadratic	Fast 

Real-Life Example: Finding Square Roots Using Newton-Raphson

Let's say you're designing a **calculator** or a **mobile app** and you want to implement a method to find square roots without using built-in functions.

You can use Newton-Raphson to compute the square root of any number A .

Problem:

Find $\sqrt{A}$, e.g., $\sqrt{25}$

We rewrite this as:

$$f(x) = x^2 - A = 0$$

Then:

$$f'(x) = 2x$$

Newton-Raphson Formula:

$$x_{n+1} = x_n - \frac{x_n^2 - A}{2x_n}$$

This simplifies to:

$$x_{n+1} = \frac{1}{2} \left(x_n + \frac{A}{x_n} \right)$$

Iteration to Find $\sqrt{25}$:

Let's start with $x_0 = 5$ (guess)

$$x_1 = \frac{1}{2} \left(5 + \frac{25}{5} \right) = \frac{1}{2}(5 + 5) = 5$$

✅ Already converged! Because 5 is the exact square root of 25.

Try a rougher guess, say $x_0 = 4$:

$$x_1 = \frac{1}{2} \left(4 + \frac{25}{4} \right) = \frac{1}{2}(4 + 6.25) = 5.125$$

$$x_2 = \frac{1}{2} \left(5.125 + \frac{25}{5.125} \right) \approx 5.000$$

✅ After just two steps, we're very close to the actual answer (5)!

Real Application:

- Used in calculators to compute square roots and inverse trigonometric functions.
 - Efficient for embedded systems with limited resources (e.g., old Nokia phones).
-

Problem Statement. Use the Newton-Raphson method to estimate the root of $f(x) = e^{-x} - x$, employing an initial guess of $x_0 = 0$.

Solution. The first derivative of the function can be evaluated as

$$f'(x) = -e^{-x} - 1$$

which can be substituted along with the original function into Eq. (6.6) to give

$$x_{i+1} = x_i - \frac{e^{-x_i} - x_i}{-e^{-x_i} - 1}$$

Starting with an initial guess of $x_0 = 0$, this iterative equation can be applied to compute

i	x_i	$\epsilon_t(\%)$
0	0	100
1	0.500000000	11.8
2	0.566311003	0.147
3	0.567143165	0.0000220
4	0.567143290	$< 10^{-8}$

Thus, the approach rapidly converges on the true root. Notice that the true percent relative error at each iteration decreases much faster than it does in simple fixed-point iteration

Gauss Elimination Method

The Gauss Elimination Method is a systematic technique for solving systems of linear equations. It reduces a system to **row-echelon form** using **elementary row operations**, then solves it using **back-substitution**.

Objectives: To solve a system of n linear equations in n variables

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n = b_2 \\ \vdots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n = b_n \end{cases}$$

Steps of the Gauss Elimination method

◆ Step 1: Forward Elimination

- Convert the coefficient matrix into **upper triangular form** by eliminating variables below the pivot (leading 1s).

◆ Step 2: Back Substitution

- Solve the last equation first and substitute back to find other unknowns.

Example: Solve The system

$$2x + 3y + z = 1$$

$$4x + 7y + 5z = 2$$

$$6x + 18y + 6z = 5$$

🔄 Step 1: Forward Elimination (with Full Detail)

We aim to convert the system into upper triangular form, where all elements below the main diagonal are zero.

Given system of equations:

$$2x + 3y + z = 1 \quad (\text{Eq1})$$

$$4x + 7y + 5z = 2 \quad (\text{Eq2})$$

$$6x + 18y + 6z = 5 \quad (\text{Eq3})$$

Step 1.1: Write the Augmented Matrix

$$\left[\begin{array}{ccc|c} 2 & 3 & 1 & 1 \\ 4 & 7 & 5 & 2 \\ 6 & 18 & 6 & 5 \end{array} \right]$$

✦ Step 1.2: Make the First Pivot = 1 (optional for clarity)

You can divide Row 1 by 2 to make the pivot element 1:

$$R_1 \leftarrow R_1 \div 2 \Rightarrow \left[\begin{array}{ccc|c} 1 & 1.5 & 0.5 & 0.5 \\ 4 & 7 & 5 & 2 \\ 6 & 18 & 6 & 5 \end{array} \right]$$

✦ Step 1.3: Eliminate the first variable (x) from Rows 2 and 3

- Eliminate Row 2:

$$R_2 \leftarrow R_2 - 4 \cdot R_1$$

$$R_2 = [4, 7, 5, 2] - 4 \cdot [1, 1.5, 0.5, 0.5] = [0, 1, 3, 0]$$

- Eliminate Row 3:

$$R_3 \leftarrow R_3 - 6 \cdot R_1$$

$$R_3 = [6, 18, 6, 5] - 6 \cdot [1, 1.5, 0.5, 0.5] = [0, 9, 3, 2]$$

Updated Matrix after First Elimination:

$$\left[\begin{array}{ccc|c} 1 & 1.5 & 0.5 & 0.5 \\ 0 & 1 & 3 & 0 \\ 0 & 9 & 3 & 2 \end{array} \right]$$

Step 1.4: Eliminate y from Row 3 (2nd column below pivot)

We want to make the second pivot the only non-zero entry in its column.

$$R_3 \leftarrow R_3 - 9 \cdot R_2 \Rightarrow [0, 9, 3, 2] - 9 \cdot [0, 1, 3, 0] = [0, 0, -24, 2]$$

Final Upper Triangular Matrix (after forward elimination):

$$\left[\begin{array}{ccc|c} 1 & 1.5 & 0.5 & 0.5 \\ 0 & 1 & 3 & 0 \\ 0 & 0 & -24 & 2 \end{array} \right]$$

This completes **Step 1: Forward Elimination**.

You can now proceed to **Step 2: Back Substitution** to solve for z , then y , then x .

Step 2: Back Substitution (Detailed)

We start with the upper triangular matrix:

$$\left[\begin{array}{ccc|c} 1 & 1.5 & 0.5 & 0.5 \\ 0 & 1 & 3 & 0 \\ 0 & 0 & -24 & 2 \end{array} \right]$$

This corresponds to the following system:

$$\text{(Eq 3)} \quad -24z = 2$$

$$\text{(Eq 2)} \quad y + 3z = 0$$

$$\text{(Eq 1)} \quad x + 1.5y + 0.5z = 0.5$$

◆ Step 2.1: Solve Eq 3 for z

$$-24z = 2 \Rightarrow z = \frac{2}{-24} = -\frac{1}{12}$$

◆ Step 2.2: Substitute z into Eq 2 to find y

$$y + 3z = 0 \Rightarrow y + 3\left(-\frac{1}{12}\right) = 0 \Rightarrow y = \frac{1}{4}$$

◆ **Step 2.3: Substitute y and z into Eq 1 to find x**

$$x + 1.5y + 0.5z = 0.5$$

Substitute values:

$$\begin{aligned}x + 1.5 \left(\frac{1}{4} \right) + 0.5 \left(-\frac{1}{12} \right) &= 0.5 \\x + \frac{3}{8} - \frac{1}{24} &= 0.5\end{aligned}$$

Common denominator: 24

$$\begin{aligned}x + \left(\frac{9}{24} - \frac{1}{24} \right) &= \frac{12}{24} \Rightarrow x + \frac{8}{24} = \frac{12}{24} \\x &= \frac{12 - 8}{24} = \frac{4}{24} = \frac{1}{6}\end{aligned} \quad x = \frac{1}{6}, \quad y = \frac{1}{4}, \quad z = -\frac{1}{12} \text{ (Ans)}$$

What We Did:

- Started with the last equation and substituted **upwards**.
- Solved each equation one-by-one for the unknowns.
- This is how **back substitution** works after Gauss Elimination.

Example: Solving for Three Unknowns (Shopping Example)

Scenario:

You bought 3 different products — pens, notebooks, and erasers — on 3 separate days with different combinations. You want to find the price of each item.

Given:

1. 2 pens + 1 notebook + 3 erasers = ₹20
2. 1 pen + 2 notebooks + 1 eraser = ₹18
3. 3 pens + 2 notebooks + 4 erasers = ₹32

Let:

- x = price of a pen
- y = price of a notebook
- z = price of an eraser

Step 1: Write as a 3×4 Augmented Matrix

$$\left[\begin{array}{ccc|c} 2 & 1 & 3 & 20 \\ 1 & 2 & 1 & 18 \\ 3 & 2 & 4 & 32 \end{array} \right]$$

Step-2: Forward Elimination

Pivot 1: Use Row 1 as the pivot

Eliminate column 1 in Row 2:

$$R2 \leftarrow R2 - 0.5 \times R1$$

$$[1, 2, 1, | 18] - 0.5 \times [2, 1, 3, | 20] = [0, 1.5, -0.5, | 8]$$

Eliminate column 1 in Row 3:

$$R3 \leftarrow R3 - 1.5 \times R1$$

$$[3, 2, 4, | 32] - 1.5 \times [2, 1, 3, | 20] = [0, 0.5, -0.5, | 2]$$

New Matrix:

$$\left[\begin{array}{ccc|c} 2 & 1 & 3 & 20 \\ 0 & 1.5 & -0.5 & 8 \\ 0 & 0.5 & -0.5 & 2 \end{array} \right]$$

- ◆ **Pivot 2: Make leading coefficient of Row 2 = 1**

$$R2 \leftarrow R2 \div 1.5$$

$$\left[0, 1, -\frac{1}{3}, \left| \frac{16}{3} \right] \right]$$

Then eliminate column 2 from Row 3:

$$R3 \leftarrow R3 - 0.5 \times R2$$

$$\left[0, 0.5, -0.5, \left| 2 \right] - 0.5 \times \left[0, 1, -\frac{1}{3}, \left| \frac{16}{3} \right] \right] = \left[0, 0, -0.333..., \left| -\frac{2}{3} \right] \right]$$

✓ Final Matrix:

$$\left[\begin{array}{ccc|c} 2 & 1 & 3 & 20 \\ 0 & 1 & -\frac{1}{3} & \frac{16}{3} \\ 0 & 0 & -\frac{1}{3} & -\frac{2}{3} \end{array} \right]$$

↔ Step 3: Back Substitution

- From Row 3:
 $-\frac{1}{3}z = -\frac{2}{3} \Rightarrow z = 2$
- From Row 2:
 $y - \frac{1}{3}(2) = \frac{16}{3} \Rightarrow y = 6$
- From Row 1:
 $2x + y + 3z = 20 \Rightarrow 2x + 6 + 6 = 20 \Rightarrow x = 4$

✓ **Final Answer:**

Item	Price (₹)
Pen (x)	4
Notebook (y)	6
Eraser (z)	2

- An augmented matrix is a matrix formed by combining the coefficient matrix of a system of linear equations with the column vector of constants on the right-hand side, separated by a vertical line
- An upper triangular matrix is a square matrix where all the elements below the main diagonal are zero.

Gauss-Jordan Elimination Method with a Real-Life Example

Gauss-Jordan Elimination is a step-by-step method used to solve a system of linear equations by transforming the augmented matrix into Reduced Row Echelon Form (RREF) using row operations.

Steps of the Method:

- Form the augmented matrix of the system.
- Make the leading entry (pivot) in each row a **1**.
- Make all other entries in the pivot's column **0**.
- Continue until the matrix is in **RREF**, then read off the solution.

✓ Real-Life Example:

✦ Scenario:

A small business sells 3 products: A, B, and C. Their total profits from 3 different sales zones are given as:

- Zone 1: $A + 2B + C = 100$
- Zone 2: $2A + 3B + 3C = 200$
- Zone 3: $A + B + 2C = 150$

Let's solve to find the individual profits per product (A, B, and C).

Step 1: Form the Augmented Matrix

$$\left[\begin{array}{ccc|c} 1 & 2 & 1 & 100 \\ 2 & 3 & 3 & 200 \\ 1 & 1 & 2 & 150 \end{array} \right]$$

◆ Step 2: Apply Gauss-Jordan Elimination

Perform row operations:

1. R1 stays the same
2. $R2 = R2 - 2 \times R1 \rightarrow [0, -1, 1, | 0]$
3. $R3 = R3 - R1 \rightarrow [0, -1, 1, | 50]$
4. $R3 = R3 - R2 \rightarrow [0, 0, 0, | 50] \rightarrow \text{✗ contradiction!}$

✦ Interpretation:

The last row implies $0 = 50$, which is not possible.

This is an **inconsistent system** — no solution exists. This suggests there is a mistake or inconsistency in the profit data.

✓ Another Real-Life Consistent Example:

Let's solve the following:

$$\begin{aligned}x + y + z &= 6 \\2x + 3y + 7z &= 20 \\x + 3y + 4z &= 13\end{aligned}$$

Augmented matrix:

$$\left[\begin{array}{ccc|c} 1 & 1 & 1 & 6 \\ 2 & 3 & 7 & 20 \\ 1 & 3 & 4 & 13 \end{array} \right]$$

After Gauss-Jordan elimination:

$$\left[\begin{array}{ccc|c} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 2 \\ 0 & 0 & 1 & 3 \end{array} \right] \quad \begin{aligned}x &= 1 \\y &= 2 \\z &= 3\end{aligned} \quad \text{(Ans)]}$$

Feature	Gauss Elimination	Gauss–Jordan Elimination
Final Form	Upper Triangular Matrix	Reduced Row Echelon Form (RREF)
Requires Back Substitution?	✓ Yes	✗ No
Simpler for Programming?	✓ Yes	✗ More complex
Faster for Large Systems?	✓ Often	✗ Slower
Direct Solution?	✗ No	✓ Yes (you don't need back-substitution)
Ideal for Inverse Finding?	✗ No	✓ Yes (Yes — Gauss–Jordan Elimination is the ideal method for finding the inverse of a matrix.)

Condition or Definition of Reduced Row Echelon Form

- Every non-zero row has a **leading 1** (first non-zero number from the left).
- If any rows consist entirely of zeros, they must be at the **bottom** of the matrix.
- The leading 1 in each row must be **to the right** of the leading 1 in the row above it.
- Each pivot (leading 1) must be the **only non-zero** entry in its column (i.e., all other values in its column are zero).

Problem Statement. Use the Gauss-Jordan technique to solve the same system as in Example 9.5:

$$\begin{aligned}3x_1 - 0.1x_2 - 0.2x_3 &= 7.85 \\0.1x_1 + 7x_2 - 0.3x_3 &= -19.3 \\0.3x_1 - 0.2x_2 + 10x_3 &= 71.4\end{aligned}$$

Solution. First, express the coefficients and the right-hand side as an augmented matrix:

$$\begin{bmatrix} 3 & -0.1 & -0.2 & 7.85 \\ 0.1 & 7 & -0.3 & -19.3 \\ 0.3 & -0.2 & 10 & 71.4 \end{bmatrix}$$

Then normalize the first row by dividing it by the pivot element, 3, to yield

$$\begin{bmatrix} 1 & -0.0333333 & -0.0666667 & 2.61667 \\ 0.1 & 7 & -0.3 & -19.3 \\ 0.3 & -0.2 & 10 & 71.4 \end{bmatrix}$$

The x_1 term can be eliminated from the second row by subtracting 0.1 times the first row from the second row. Similarly, subtracting 0.3 times the first row from the third row will eliminate the x_1 term from the third row:

$$\begin{bmatrix} 1 & -0.0333333 & -0.0666667 & 2.61667 \\ 0 & 7.00333 & -0.293333 & -19.5617 \\ 0 & -0.190000 & 10.0200 & 70.6150 \end{bmatrix}$$

Next, normalize the second row by dividing it by 7.00333:

$$\begin{bmatrix} 1 & -0.0333333 & -0.0666667 & 2.61667 \\ 0 & 1 & -0.0418848 & -2.79320 \\ 0 & -0.190000 & 10.0200 & 70.6150 \end{bmatrix}$$

Reduction of the x_2 terms from the first and third equations gives

$$\begin{bmatrix} 1 & 0 & -0.0680629 & 2.52356 \\ 0 & 1 & -0.0418848 & -2.79320 \\ 0 & 0 & 10.01200 & 70.0843 \end{bmatrix}$$

The third row is then normalized by dividing it by 10.0120:

$$\begin{bmatrix} 1 & 0 & -0.0680629 & 2.52356 \\ 0 & 1 & -0.0418848 & -2.79320 \\ 0 & 0 & 1 & 7.0000 \end{bmatrix}$$

Finally, the x_3 terms can be reduced from the first and the second equations to give

$$\begin{bmatrix} 1 & 0 & 0 & 3.0000 \\ 0 & 1 & 0 & -2.5000 \\ 0 & 0 & 1 & 7.0000 \end{bmatrix}$$

$$x_1 = 3, x_2 = -2.5, \text{ and } x_3 = 7$$

Interpolation: Diagonal and horizontal differences

- In interpolation, particularly in finite differences or difference tables, we often compute diagonal and horizontal differences to estimate the value of a function between known data points.

1. Horizontal Differences (First-order differences)

These are the differences **along the rows** of a table (i.e., between successive values of a function):

$$\Delta y_i = y_{i+1} - y_i$$

For example, given function values:

x	y
1	2
2	4
3	7
4	11

Then the **first-order horizontal differences** are:

$$\Delta y_0 = 4 - 2 = 2$$

$$\Delta y_1 = 7 - 4 = 3$$

$$\Delta y_2 = 11 - 7 = 4$$

These can be used to construct a **difference table**.

2. Diagonal Differences (Higher-order differences)

These are the differences taken **diagonally**, i.e., between the horizontal differences.

Second-order (diagonal) differences:

$$\Delta^2 y_0 = \Delta y_1 - \Delta y_0 = 3 - 2 = 1$$

$$\Delta^2 y_1 = \Delta y_2 - \Delta y_1 = 4 - 3 = 1$$

Third-order difference:

$$\Delta^3 y_0 = \Delta^2 y_1 - \Delta^2 y_0 = 1 - 1 = 0$$

So the full **difference table** looks like:

x	y	Δy	$\Delta^2 y$	$\Delta^3 y$
1	2	2	1	0
2	4	3	1	
3	7	4		
4	11			

✓ Purpose in Interpolation:

These differences are used in **Newton's forward and backward interpolation** formulas.

📊 Newton's Forward Formula (for evenly spaced x-values):

$$f(x) \approx y_0 + u\Delta y_0 + \frac{u(u-1)}{2!}\Delta^2 y_0 + \frac{u(u-1)(u-2)}{3!}\Delta^3 y_0 + \dots$$

Where:

- $u = \frac{x-x_0}{h}$
- h is the interval between x -values

⚠ Summary

Type	What it does	Used in
Horizontal diff	First-order changes (Δy)	Start of difference table
Diagonal diff	Higher-order changes ($\Delta^2 y$, $\Delta^3 y$, ...)	Newton interpolation

Differences of a polynomial

- For a polynomial of degree n , the n^{th} order finite difference is constant, and all higher-order differences are zero.
- Finite differences (forward, backward, etc.) computed from the function values of the polynomial at evenly spaced points.

✓ Example: Polynomial of Degree 2

Let's take:

$$f(x) = x^2 + 1$$

Evaluate at evenly spaced x values:

x	$f(x)$
0	$0^2 + 1 = 1$
1	$1^2 + 1 = 2$
2	$2^2 + 1 = 5$
3	$3^2 + 1 = 10$
4	$4^2 + 1 = 17$

First Differences (Δ):

$$\Delta f_0 = f(1) - f(0) = 2 - 1 = 1$$
$$\Delta f_1 = f(2) - f(1) = 5 - 2 = 3$$
$$\Delta f_2 = f(3) - f(2) = 10 - 5 = 5$$
$$\Delta f_3 = f(4) - f(3) = 17 - 10 = 7$$

Second Differences (Δ^2):

$$\Delta^2 f_0 = 3 - 1 = 2$$
$$\Delta^2 f_1 = 5 - 3 = 2$$
$$\Delta^2 f_2 = 7 - 5 = 2$$

The second differences are **constant**, which confirms this is a **degree 2 polynomial**.

Third Differences (Δ^3):

$$\Delta^3 f_0 = 2 - 2 = 0$$
$$\Delta^3 f_1 = 2 - 2 = 0$$

All higher differences are **zero**, as expected.

Summary Table:

x	$f(x)$	Δ	Δ^2	Δ^3
0	1	1	2	0
1	2	3	2	
2	5	5		
3	10	7		
4	17			

Key Insight:

Polynomial Degree	First Constant Difference
Linear (degree 1)	First difference constant
Quadratic (deg 2)	Second difference constant
Cubic (deg 3)	Third difference constant

Newton's Forward Interpolation Formula

Use when:

- Data points are equally spaced
- You're interpolating **near the beginning** of the data

Formula:

$$f(x) = y_0 + u\Delta y_0 + \frac{u(u-1)}{2!}\Delta^2 y_0 + \frac{u(u-1)(u-2)}{3!}\Delta^3 y_0 + \dots$$

Where:

- x_0 is the first known value of x
- h is the spacing: $h = x_1 - x_0$
- $u = \frac{x-x_0}{h}$
- $\Delta y_0, \Delta^2 y_0, \dots$ are forward differences from a difference table

Full step-by-step process of using Newton's Forward Interpolation to find $f(2.5)$.

✓ Step 1: Given Data

We'll use the following table (based on $f(x) = x^3$):

x	$f(x) = y$
1	1
2	8
3	27
4	64

We want to find $f(2.5)$

✓ Step 2: Check Conditions

- Even spacing: $h = x_1 - x_0 = 2 - 1 = 1$ ✓
- Use **forward interpolation** since 2.5 is closer to the **beginning** of the table.

✓ Step 3: Create the Forward Difference Table

x	y	Δy	$\Delta^2 y$	$\Delta^3 y$
1	1	7	12	6
2	8	19	18	
3	27	37		
4	64			

How the differences were computed:

- $\Delta y_0 = 8 - 1 = 7$
- $\Delta y_1 = 27 - 8 = 19$
- $\Delta y_2 = 64 - 27 = 37$
- $\Delta^2 y_0 = 19 - 7 = 12$
- $\Delta^2 y_1 = 37 - 19 = 18$
- $\Delta^3 y_0 = 18 - 12 = 6$

✔ Step 4: Use Newton's Forward Interpolation Formula

$$f(x) = y_0 + u\Delta y_0 + \frac{u(u-1)}{2!}\Delta^2 y_0 + \frac{u(u-1)(u-2)}{3!}\Delta^3 y_0$$

Where:

- $x_0 = 1$
- $h = 1$
- $u = \frac{x-x_0}{h} = \frac{2.5-1}{1} = 1.5$

✔ Step 5: Plug in values

$$\begin{aligned} f(2.5) &= y_0 + u\Delta y_0 + \frac{u(u-1)}{2!}\Delta^2 y_0 + \frac{u(u-1)(u-2)}{3!}\Delta^3 y_0 \\ &= 1 + (1.5)(7) + \frac{1.5(0.5)}{2}(12) + \frac{1.5(0.5)(-0.5)}{6}(6) \end{aligned}$$

✔ Step 6: Calculate Each Term

- First term: 1
- Second term: $1.5 \times 7 = 10.5$
- Third term: $\frac{1.5 \times 0.5}{2} \times 12 = \frac{0.75}{2} \times 12 = 0.375 \times 12 = 4.5$
- Fourth term: $\frac{1.5 \times 0.5 \times (-0.5)}{6} \times 6 = \frac{-0.375}{6} \times 6 = -0.375$

✔ Step 7: Final Result

$$f(2.5) = 1 + 10.5 + 4.5 - 0.375 = \boxed{15.625}$$

✔ Check (for comparison):

Since the actual function is $f(x) = x^3$:

$$f(2.5) = (2.5)^3 = 15.625$$

- ✔ The interpolation result matches the exact value — success!

Newton's Backward Interpolation Formula to compute: $f(3.5)$ Using the same data table:

✓ Step 1: Given Table

x	$f(x) = y$
1	1
2	8
3	27
4	64

We'll interpolate **near the end** of the table → use **backward interpolation**.

✓ Step 2: Check spacing

- $h = x_1 - x_0 = 1$
- $x_n = 4$
- $u = \frac{x - x_n}{h} = \frac{3.5 - 4}{1} = -0.5$

✓ Step 3: Build Backward Difference Table

x	y	∇y	$\nabla^2 y$	$\nabla^3 y$
1	1			
2	8	7		
3	27	19	12	
4	64	37	18	6

How we computed:

- $\nabla y_3 = 64 - 27 = 37$
- $\nabla y_2 = 27 - 8 = 19$
- $\nabla y_1 = 8 - 1 = 7$
- $\nabla^2 y_3 = 37 - 19 = 18$
- $\nabla^2 y_2 = 19 - 7 = 12$
- $\nabla^3 y_3 = 18 - 12 = 6$

✓ Step 4: Newton's Backward Interpolation Formula

$$f(x) = y_n + u\nabla y_n + \frac{u(u+1)}{2!}\nabla^2 y_n + \frac{u(u+1)(u+2)}{3!}\nabla^3 y_n + \dots$$

Where:

- $x_n = 4$
- $y_n = f(4) = 64$
- $u = -0.5$

✓ Step 5: Plug in values

$$\begin{aligned} f(3.5) &= y_n + u\nabla y_n + \frac{u(u+1)}{2}\nabla^2 y_n + \frac{u(u+1)(u+2)}{6}\nabla^3 y_n \\ &= 64 + (-0.5)(37) + \frac{-0.5(0.5)}{2}(18) + \frac{-0.5(0.5)(1.5)}{6}(6) \end{aligned}$$

✓ Step 6: Evaluate Each Term

- First: 64
- Second: $-0.5 \times 37 = -18.5$
- Third: $\frac{-0.25}{2} \times 18 = -0.125 \times 18 = -2.25$
- Fourth: $\frac{-0.5 \times 0.5 \times 1.5}{6} \times 6 = \frac{-0.375}{6} \times 6 = -0.375$

Step 7: Final Calculation

$$f(3.5) = 64 - 18.5 - 2.25 - 0.375 = \boxed{42.875}$$

✓ Check (Optional):

True value from $f(x) = x^3$:

$$f(3.5) = (3.5)^3 = 42.875$$

✓ Perfect match! Our backward interpolation is accurate.

Comparison between Newton's Forward and Backward Interpolation methods

Feature	Forward Interpolation	Backward Interpolation
Use case	When x is near the beginning of the data	When x is near the end of the data
Reference point	First value x_0	Last value x_n
Variable u	$u = \frac{x-x_0}{h}$	$u = \frac{x-x_n}{h}$
Differences used	Forward differences (Δy)	Backward differences (∇y)
General formula	$f(x) = y_0 + u\Delta y_0 + \frac{u(u-1)}{2!}\Delta^2 y_0 + \dots$	$f(x) = y_n + u\nabla y_n + \frac{u(u+1)}{2!}\nabla^2 y_n + \dots$
Best suited for	Values of x close to x_0	Values of x close to x_n
Direction of difference table	From top to bottom (forward steps)	From bottom to top (backward steps)

Problem

A manufacturer measures the temperature of a chemical reaction every 10 seconds:

Time (seconds)	Temperature (°C)
0	20.0
10	22.5
20	27.0
30	33.5
40	41.0

Estimate the temperature at $t = 15$ seconds using Newton's Forward Interpolation.

Step 1: Create the finite difference table

Let's denote:

- $x_0 = 0$
- $h = 10$ (equal interval)
- $y_0 = 20.0$

Now, compute forward differences:

x (sec)	y	Δy	$\Delta^2 y$	$\Delta^3 y$	$\Delta^4 y$
0	20.0	2.5	2.0	0.5	0.0
10	22.5	4.5	2.5	0.5	
20	27.0	6.5	3.0		
30	33.5	7.5			
40	41.0				

Step 2: Use Newton's Forward Interpolation Formula

Newton's forward formula is:

$$f(x) = y_0 + u\Delta y_0 + \frac{u(u-1)}{2!}\Delta^2 y_0 + \frac{u(u-1)(u-2)}{3!}\Delta^3 y_0 + \dots$$

Where:

- $u = \frac{x-x_0}{h} = \frac{15-0}{10} = 1.5$

Step 3: Substitute values

$$\begin{aligned} f(15) &= 20.0 + 1.5(2.5) + \frac{1.5(0.5)}{2}(2.0) + \frac{1.5(0.5)(-0.5)}{6}(0.5) \\ &= 20.0 + 3.75 + 0.75 + (-0.03125) \\ &\approx \boxed{24.47^\circ\text{C}} \end{aligned}$$

Final Answer:

At 15 seconds, the estimated temperature is approximately **24.47°C**.

Newton-Cotes Integration Formulas

- Newton-Cotes Integration Formulas are numerical integration techniques that approximate the definite integral of a function using polynomials passing through equally spaced points.
- These formulas are a family of rules derived from interpolating polynomials.

General Idea:

Given a function $f(x)$ on interval $[a, b]$, divide it into n equal subintervals. Use polynomial interpolation to approximate $f(x)$, then integrate the polynomial instead.

◆ General Newton-Cotes Formula:

$$\int_a^b f(x) dx \approx \sum_{i=0}^n w_i f(x_i)$$

- $x_0, x_1, \dots, x_n$ are equally spaced points in $[a, b]$
- w_i are **weights** depending on the interpolation
- The **degree** n of the formula is the number of points used minus 1.

◆ Types of Newton-Cotes Formulas:

✓ 1. Closed Newton-Cotes (includes endpoints)

- Uses $n + 1$ points: $x_0 = a, x_1, \dots, x_n = b$

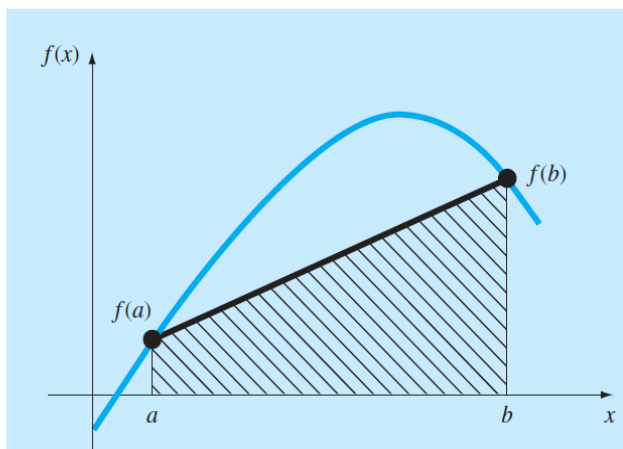
Examples:

Name	Formula
Trapezoidal Rule (n=1)	$\int_a^b f(x) dx \approx \frac{b-a}{2} [f(a) + f(b)]$
Simpson's 1/3 Rule (n=2)	$\int_a^b f(x) dx \approx \frac{b-a}{6} \left[f(a) + 4f\left(\frac{a+b}{2}\right) + f(b) \right]$
Simpson's 3/8 Rule (n=3)	$\int_a^b f(x) dx \approx \frac{3h}{8} [f(x_0) + 3f(x_1) + 3f(x_2) + f(x_3)]$
Boole's Rule (n=4)	$\int_a^b f(x) dx \approx \frac{2h}{45} [7f(x_0) + 32f(x_1) + 12f(x_2) + 32f(x_3) + 7f(x_4)]$

Where $h = \frac{b-a}{n}$

FIGURE 21.4

Graphical depiction of the trapezoidal rule.



THE TRAPEZOIDAL RULE

The *trapezoidal rule* is the first of the Newton-Cotes closed integration formulas. It corresponds to the case where the polynomial in Eq. (21.1) is first order:

$$I = \int_a^b f(x) dx \cong \int_a^b f_1(x) dx$$

Recall from Chap. 18 that a straight line can be represented as [Eq. (18.2)]

$$f_1(x) = f(a) + \frac{f(b) - f(a)}{b - a}(x - a) \quad (21.2)$$

The area under this straight line is an estimate of the integral of $f(x)$ between the limits a and b :

$$I = \int_a^b \left[f(a) + \frac{f(b) - f(a)}{b - a}(x - a) \right] dx$$

The result of the integration (see Box 21.1 for details) is

$$I = (b - a) \frac{f(a) + f(b)}{2} \quad (21.3)$$

which is called the *trapezoidal rule*.

Box 21.1 Derivation of Trapezoidal Rule

Before integration, Eq. (21.2) can be expressed as

$$f_1(x) = \frac{f(b) - f(a)}{b - a}x + f(a) - \frac{af(b) - af(a)}{b - a}$$

Grouping the last two terms gives

$$f_1(x) = \frac{f(b) - f(a)}{b - a}x + \frac{bf(a) - af(a) - af(b) + af(a)}{b - a}$$

or

$$f_1(x) = \frac{f(b) - f(a)}{b - a}x + \frac{bf(a) - af(b)}{b - a}$$

which can be integrated between $x = a$ and $x = b$ to yield

$$I = \frac{f(b) - f(a)}{b - a} \frac{x^2}{2} + \frac{bf(a) - af(b)}{b - a} x \Big|_a^b$$

This result can be evaluated to give

$$I = \frac{f(b) - f(a)}{b - a} \frac{(b^2 - a^2)}{2} + \frac{bf(a) - af(b)}{b - a} (b - a)$$

Now, since $b^2 - a^2 = (b - a)(b + a)$,

$$I = [f(b) - f(a)] \frac{b + a}{2} + bf(a) - af(b)$$

Multiplying and collecting terms yields

$$I = (b - a) \frac{f(a) + f(b)}{2}$$

which is the formula for the trapezoidal rule.

Single Application of the Trapezoidal Rule

Problem Statement. Use Eq. (21.3) to numerically integrate

$$f(x) = 0.2 + 25x - 200x^2 + 675x^3 - 900x^4 + 400x^5$$

from $a = 0$ to $b = 0.8$. Recall from Sec. PT6.2 that the exact value of the integral can be determined analytically to be 1.640533.

Solution. The function values

$$f(0) = 0.2$$

$$f(0.8) = 0.232$$

can be substituted into Eq. (21.3) to yield

$$I \cong 0.8 \frac{0.2 + 0.232}{2} = 0.1728$$

which represents an error of

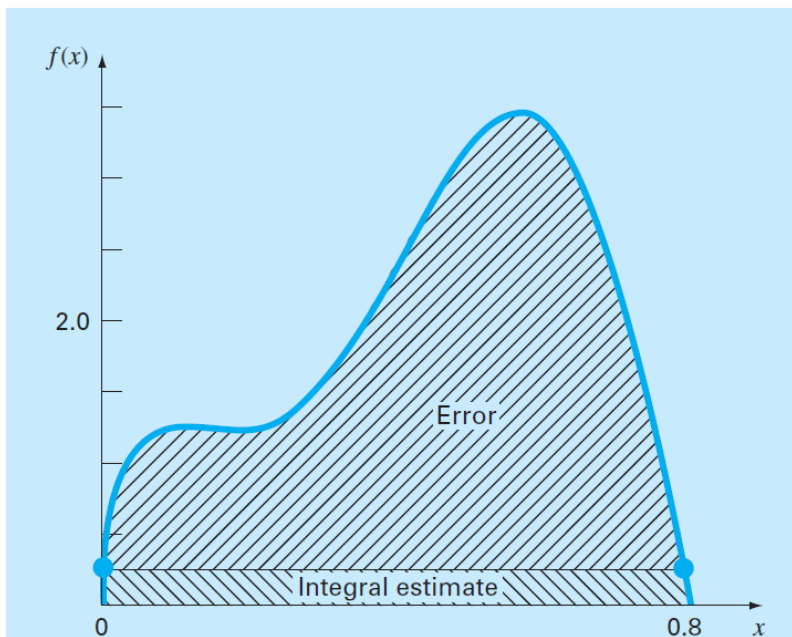
$$E_t = 1.640533 - 0.1728 = 1.467733$$

which corresponds to a percent relative error of $\varepsilon_t = 89.5\%$. The reason for this large error is evident from the graphical depiction in Fig. 21.6. Notice that the area under the straight line neglects a significant portion of the integral lying above the line.

Open The pdf for rest of the part if needed

FIGURE 21.6

Graphical depiction of the use of a single application of the trapezoidal rule to approximate the integral of $f(x) = 0.2 + 25x - 200x^2 + 675x^3 - 900x^4 + 400x^5$ from $x = 0$ to 0.8 .



21.2.1 Simpson's 1/3 Rule

Simpson's 1/3 rule results when a second-order interpolating polynomial is substituted into Eq. (21.1):

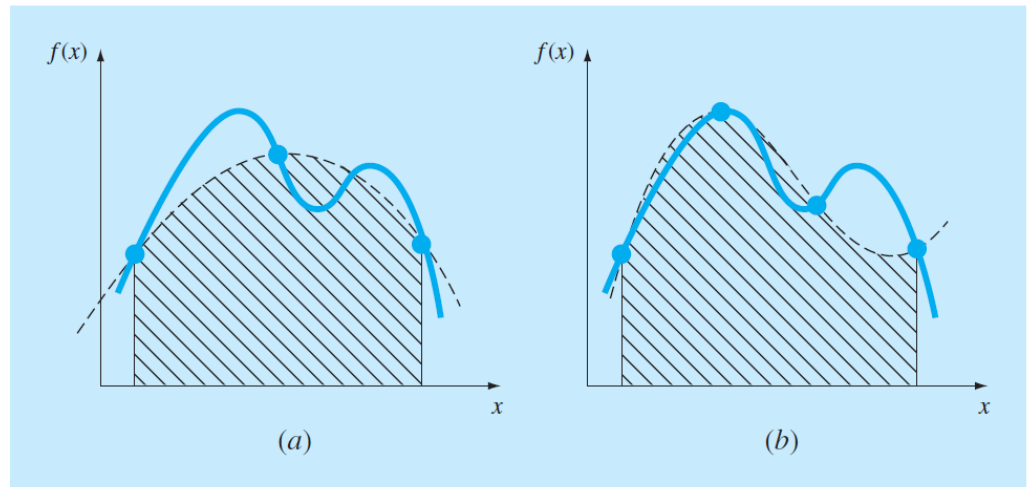
$$I = \int_a^b f(x) dx \cong \int_a^b f_2(x) dx$$

If a and b are designated as x_0 and x_2 and $f_2(x)$ is represented by a second-order Lagrange polynomial [Eq. (18.23)], the integral becomes

$$I = \int_{x_0}^{x_2} \left[\frac{(x - x_1)(x - x_2)}{(x_0 - x_1)(x_0 - x_2)} f(x_0) + \frac{(x - x_0)(x - x_2)}{(x_1 - x_0)(x_1 - x_2)} f(x_1) + \frac{(x - x_0)(x - x_1)}{(x_2 - x_0)(x_2 - x_1)} f(x_2) \right] dx$$

FIGURE 21.10

(a) Graphical depiction of Simpson's 1/3 rule: It consists of taking the area under a parabola connecting three points. (b) Graphical depiction of Simpson's 3/8 rule: It consists of taking the area under a cubic equation connecting four points.



Single Application of Simpson's 1/3 Rule

Problem Statement. Use Eq. (21.15) to integrate

$$f(x) = 0.2 + 25x - 200x^2 + 675x^3 - 900x^4 + 400x^5$$

from $a = 0$ to $b = 0.8$. Recall that the exact integral is 1.640533.

Solution.

$$f(0) = 0.2 \quad f(0.4) = 2.456 \quad f(0.8) = 0.232$$

Therefore, Eq. (21.15) can be used to compute

$$I \cong 0.8 \frac{0.2 + 4(2.456) + 0.232}{6} = 1.367467$$

which represents an exact error of

$$E_t = 1.640533 - 1.367467 = 0.2730667 \quad \epsilon_t = 16.6\%$$

which is approximately 5 times more accurate than for a single application of the trapezoidal rule (Example 21.1).

21.2.3 Simpson's 3/8 Rule

In a similar manner to the derivation of the trapezoidal and Simpson's 1/3 rule, a third-order Lagrange polynomial can be fit to four points and integrated:

$$I = \int_a^b f(x) dx \cong \int_a^b f_3(x) dx$$

to yield

$$I \cong \frac{3h}{8} [f(x_0) + 3f(x_1) + 3f(x_2) + f(x_3)]$$

where $h = (b - a)/3$. This equation is called *Simpson's 3/8 rule* because h is multiplied by $3/8$. It is the third Newton-Cotes closed integration formula. The 3/8 rule can also be expressed in the form of Eq. (21.5):

$$I \cong \underbrace{(b - a)}_{\text{Width}} \underbrace{\frac{f(x_0) + 3f(x_1) + 3f(x_2) + f(x_3)}{8}}_{\text{Average height}} \quad (21.20)$$

Simpson's 3/8 Rule

Problem Statement.

(a) Use Simpson's 3/8 rule to integrate

$$f(x) = 0.2 + 25x - 200x^2 + 675x^3 - 900x^4 + 400x^5$$

from $a = 0$ to $b = 0.8$.

(b) Use it in conjunction with Simpson's 1/3 rule to integrate the same function for five segments.

Solution.

(a) A single application of Simpson's 3/8 rule requires four equally spaced points:

$$f(0) = 0.2 \quad f(0.2667) = 1.432724$$

$$f(0.5333) = 3.487177 \quad f(0.8) = 0.232$$

Using Eq. (21.20),

$$I \cong 0.8 \frac{0.2 + 3(1.432724 + 3.487177) + 0.232}{8} = 1.519170$$

$$E_t = 1.640533 - 1.519170 = 0.1213630 \quad \epsilon_t = 7.4\%$$

$$E_a = -\frac{(0.8)^5}{6480}(-2400) = 0.1213630$$

(b) The data needed for a five-segment application ($h = 0.16$) is

$$\begin{aligned} f(0) &= 0.2 & f(0.16) &= 1.296919 \\ f(0.32) &= 1.743393 & f(0.48) &= 3.186015 \\ f(0.64) &= 3.181929 & f(0.80) &= 0.232 \end{aligned}$$

The integral for the first two segments is obtained using Simpson's 1/3 rule:

$$I \cong 0.32 \frac{0.2 + 4(1.296919) + 1.743393}{6} = 0.3803237$$

For the last three segments, the 3/8 rule can be used to obtain

$$I \cong 0.48 \frac{1.743393 + 3(3.186015 + 3.181929) + 0.232}{8} = 1.264754$$

The total integral is computed by summing the two results:

$$\begin{aligned} I &= 0.3803237 + 1.264753 = 1.645077 \\ E_t &= 1.640533 - 1.645077 = -0.00454383 \quad \varepsilon_t = -0.28\% \end{aligned}$$

Ordinary differential equations

What is an Ordinary Differential Equation?

An **Ordinary Differential Equation (ODE)** is an equation that relates a **function** with its **derivatives**.

- It describes how a quantity changes over time (or another variable).
- It is "ordinary" because it involves **derivatives with respect to a single variable** (usually time t or space x).

General Form:

Where:

$$\frac{dy}{dx} = f(x, y)$$

- y is the unknown function
- x is the independent variable (e.g., time)
- $f(x, y)$ defines how y changes

Real Life Example of Ordinary Differential Equations:

Application	ODE Used
Population growth	$\frac{dP}{dt} = rP$ (exponential)
Radioactive decay	$\frac{dN}{dt} = -\lambda N$

Newton's Law	$F = ma = m \frac{d^2x}{dt^2}$
Charging a capacitor	$\frac{dV}{dt} + \frac{1}{RC} V = \frac{V_0}{RC}$
Pendulum motion	$\frac{d^2\theta}{dt^2} + \frac{g}{l} \sin \theta = 0$

Euler's method-Simplest Numerical Method for Solving ODEs

✔ What Is Euler's Method?

Euler's method is a numerical technique to approximate the solution of ordinary differential equations (ODEs).

Given:

dy/dx = f(x, y), y(x0) = y0

Euler's method estimates the value of y at the next point using the slope at the current point.

1234 Euler's Formula:

yn+1 = yn + h · f(xn, yn)

Where:

- (xn, yn) is the current point
- h is the step size
- f(x, y) is the derivative function
- yn+1 is the estimated value at xn+1 = xn + h

Solve:

dy/dx = x + y, y(0) = 1

Use step size h = 0.1, compute up to x = 0.3

Step-by-Step:

Step	x_n	y_n	$f(x_n, y_n) = x + y$	$y_{n+1} = y_n + h \cdot f$
0	0.0	1.0000	0 + 1 = 1.0	1.0 + 0.1×1.0 = 1.1000
1	0.1	1.1000	0.1 + 1.1 = 1.2	1.1 + 0.1×1.2 = 1.2200
2	0.2	1.2200	0.2 + 1.22 = 1.42	1.22 + 0.1×1.42 = 1.3620
3	0.3	1.3620	—	Stop here

✔ Final Answer: y(0.3) ≈ 1.362

Summary

Feature	Euler's Method
Type	Numerical method for ODEs
Formula	$y_{n+1} = y_n + h \cdot f(x_n, y_n)$
Accuracy	Low (improves with smaller step size)
Best for	Simple ODEs or quick approximations

Accuracy:

Simple and fast ✓

But **not very accurate** ✗ for large step sizes

Better methods: **Heun's**, **Runge-Kutta**, etc.

Runge-Kutta Methods

This chapter is devoted to solving ordinary differential equations of the form

$$\frac{dy}{dx} = f(x, y)$$

In Chap. 1, we used a numerical method to solve such an equation for the velocity of the falling parachutist. Recall that the method was of the general form

$$\text{New value} = \text{old value} + \text{slope} \times \text{step size}$$

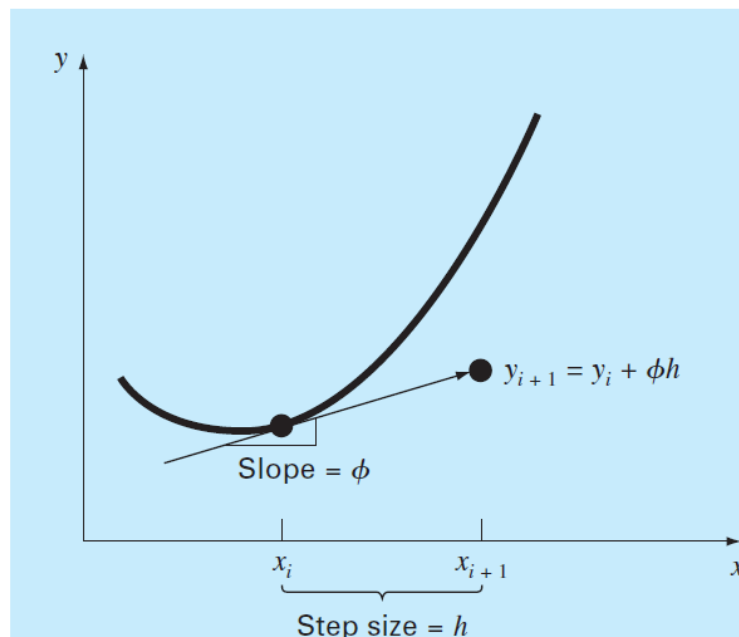
or, in mathematical terms,

$$y_{i+1} = y_i + \phi h \quad (25.1)$$

According to this equation, the slope estimate of ϕ is used to extrapolate from an old value y_i to a new value y_{i+1} over a distance h (Fig. 25.1). This formula can be applied step by step to compute out into the future and, hence, trace out the trajectory of the solution.

FIGURE 25.1

Graphical depiction of a one-step method.



EULER'S METHOD

The first derivative provides a direct estimate of the slope at x_i (Fig. 25.2):

$$\phi = f(x_i, y_i)$$

where $f(x_i, y_i)$ is the differential equation evaluated at x_i and y_i . This estimate can be substituted into Eq. (25.1):

$$y_{i+1} = y_i + f(x_i, y_i)h \quad (25.2)$$

This formula is referred to as *Euler's* (or the *Euler-Cauchy* or the *point-slope method*). A new value of y is predicted using the slope (equal to the first derivative at the original value of x) to extrapolate linearly over the step size h (Fig. 25.2).

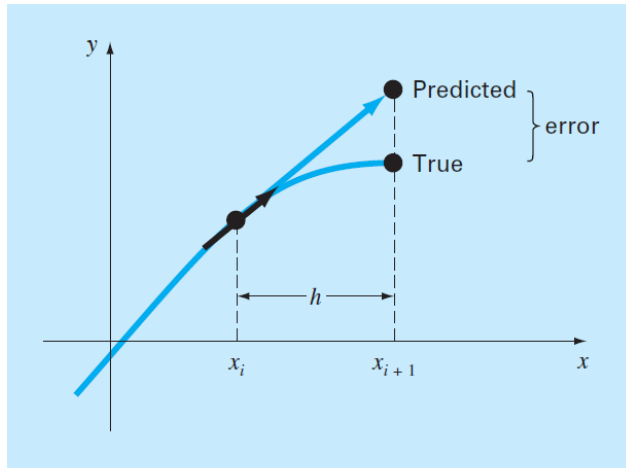


FIGURE 25.2

Euler's Method

Problem Statement. Use Euler's method to numerically integrate Eq. (PT7.13):

$$\frac{dy}{dx} = -2x^3 + 12x^2 - 20x + 8.5$$

from $x = 0$ to $x = 4$ with a step size of 0.5. The initial condition at $x = 0$ is $y = 1$. Recall that the exact solution is given by Eq. (PT7.16):

$$y = -0.5x^4 + 4x^3 - 10x^2 + 8.5x + 1$$

Solution. Equation (25.2) can be used to implement Euler's method:

$$y(0.5) = y(0) + f(0, 1)0.5$$

where $y(0) = 1$ and the slope estimate at $x = 0$ is

$$f(0, 1) = -2(0)^3 + 12(0)^2 - 20(0) + 8.5 = 8.5$$

Therefore,

$$y(0.5) = 1.0 + 8.5(0.5) = 5.25$$

The true solution at $x = 0.5$ is

$$y = -0.5(0.5)^4 + 4(0.5)^3 - 10(0.5)^2 + 8.5(0.5) + 1 = 3.21875$$

Thus, the error is

$$E_t = \text{true} - \text{approximate} = 3.21875 - 5.25 = -2.03125$$

or, expressed as percent relative error, $\varepsilon_t = -63.1\%$. For the second step,

$$\begin{aligned} y(1) &= y(0.5) + f(0.5, 5.25)0.5 \\ &= 5.25 + [-2(0.5)^3 + 12(0.5)^2 - 20(0.5) + 8.5]0.5 \\ &= 5.875 \end{aligned}$$

The true solution at $x = 1.0$ is 3.0, and therefore, the percent relative error is -95.8% . The computation is repeated, and the results are compiled in Table 25.1 and Fig. 25.3.

TABLE 25.1 Comparison of true and approximate values of the integral of $y' = -2x^3 + 12x^2 - 20x + 8.5$, with the initial condition that $y = 1$ at $x = 0$. The approximate values were computed using Euler's method with a step size of 0.5. The local error refers to the error incurred over a single step. It is calculated with a Taylor series expansion as in Example 25.2. The global error is the total discrepancy due to past as well as present steps.

x	y_{true}	y_{Euler}	Percent Relative Error	
			Global	Local
0.0	1.00000	1.00000		
0.5	3.21875	5.25000	-63.1	-63.1
1.0	3.00000	5.87500	-95.8	-28.1
1.5	2.21875	5.12500	-131.0	-1.4
2.0	2.00000	4.50000	-125.0	20.3
2.5	2.71875	4.75000	-74.7	17.2
3.0	4.00000	5.87500	-46.9	3.9
3.5	4.71875	7.12500	-51.0	-11.3
4.0	3.00000	7.00000	-133.3	-53.1

Real-Life Incidence Example: **Vehicle Motion with Changing Speed**

Scenario:

A car is moving on a road. Its **acceleration** is not constant, but depends on time. You only know:

$$\frac{dv}{dt} = a(t) = 2t$$

That means the acceleration **increases with time** — e.g., the car accelerates faster as time goes on.

You are asked to estimate the **velocity** of the car at $t = 0.4$ seconds, given:

- Initial velocity $v(0) = 0$ (starts from rest)
- Use Euler's Method with step size $h = 0.1$

Solution:

Step-by-Step Using Euler’s Method:

We solve the ODE:

$$\frac{dv}{dt} = 2t$$

Initial value:

$$v_0 = 0, \quad t_0 = 0$$

Formula:

$$v_{n+1} = v_n + h \cdot f(t_n)$$

Iterations:

Step	t_n	v_n	$f(t_n) = 2t$	$v_{n+1} = v_n + h \cdot f$
0	0.0	0.000	0.0	$0.000 + 0.1 \times 0.0 = 0.000$
1	0.1	0.000	0.2	$0.000 + 0.1 \times 0.2 = 0.020$
2	0.2	0.020	0.4	$0.020 + 0.1 \times 0.4 = 0.060$
3	0.3	0.060	0.6	$0.060 + 0.1 \times 0.6 = 0.120$
4	0.4	0.120	0.8	$0.120 + 0.1 \times 0.8 = \mathbf{0.200}$

✔ Estimated velocity at $t = 0.4$ seconds is approximately 0.200 m/s

Real Meaning:

In physics or engineering:

- You don’t always have exact formulas.
- You often have **differential equations** from sensors or simulations.
- Euler’s method lets you **predict system behavior over time**, step by step.

Other Real-Life Applications of Euler’s Method:

Domain	Application Example
 Medical	Blood flow rate modeling in arteries
 Environment	Predicting pollution spread in a river over time
 Engineering	Modeling temperature in a rod (heat flow)
 Aerospace	Simulating rocket trajectory in early flight seconds
 Chemistry	Rate of reaction change (kinetics)

Least-Squares Regression

What is Least-Squares Regression?

Least-Squares Regression is a method used to **find the best-fitting line** (or curve) through a set of data points by **minimizing the sum of the squared errors** (differences between actual and predicted values).

Goal:

Minimize the total squared error:

$$\text{Loss} = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Where:

- y_i = actual value
- $\hat{y}_i$ = predicted value from model

This is also called **Ordinary Least Squares (OLS)**.

Types of Least-Squares Regression

Type	Description	Use Case	
Linear Least Squares	Fits a straight line $Y = \beta_0 + \beta_1 X$	When relationship is linear	
Multiple Linear Regression	Line fits multiple inputs: $Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots$	Predict house price based on area, rooms, location	
Polynomial Regression	Fits nonlinear curves: $Y = a + bX + cX^2 + \dots$	U-shaped or curved trends	
Nonlinear Least Squares	Model is non-linear in parameters (e.g., exponential)	Population growth, physics models	
Weighted Least Squares (WLS)	Gives weights to data points (some errors count more)	When some observations are more reliable	
Ridge / Lasso Regression	Penalizes complexity to prevent overfitting	When multicollinearity or overfitting is a concern	
Total Least Squares	Minimizes errors in both X and Y directions	Used in image processing or when X values have noise	

Linear Regression

What is Linear Regression?

Linear Regression is a supervised machine learning algorithm that predicts a continuous output based on the linear relationship between input (independent) and output (dependent) variables.

Equation:

$$Y = \beta_0 + \beta_1 X + \epsilon$$

Where:

- Y = Output (dependent variable)
- X = Input (independent variable)
- β_0 = Intercept (value of Y when $X=0$)
- β_1 = Slope (change in Y per unit change in X)

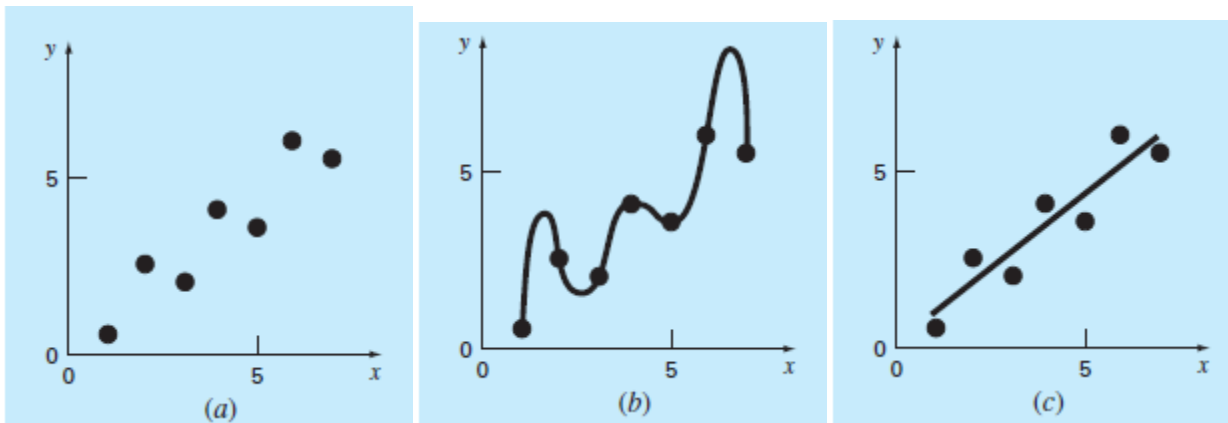


FIGURE 17.1 (a) Data exhibiting significant error. (b) Polynomial fit oscillating beyond the range of the data. (c) More satisfactory result using the least-squares fit.

Training a linear regression model using Gradient Descent

Goal: Train a linear model to fit this data

Example Dataset:	x (input)	y(output)
	1	2
	2	4
	3	6

We want to learn: $\hat{y} = mx + b$

So that it predicts output y as close as possible to the actual values.

Step-by-Step Gradient Descent

Step 1: Initialize

Start with random values:

- $m=0$
- $b=0$
- Learning rate $\alpha=0.01$
- ◆ Step 2: Define the Loss (Error)

Use Mean Squared Error (MSE):

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \frac{1}{3} [(2 - \hat{y}_1)^2 + (4 - \hat{y}_2)^2 + (6 - \hat{y}_3)^2]$$

Where:

$$\hat{y}_i = m \cdot x_i + b$$

◆ Step 3: Compute Gradients

We update parameters using partial derivatives:

$$\begin{aligned}\frac{\partial \text{Loss}}{\partial m} &= -\frac{2}{n} \sum (x_i)(y_i - \hat{y}_i) \\ \frac{\partial \text{Loss}}{\partial b} &= -\frac{2}{n} \sum (y_i - \hat{y}_i)\end{aligned}$$

The Model:

We're trying to fit a line:

$$\hat{y}_i = m \cdot x_i + b$$

Where:

- x_i = input
- y_i = actual output
- $\hat{y}_i$ = predicted output
- m = slope (weight)
- b = intercept (bias)

◆ The Loss Function (MSE):

$$\text{Loss} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \frac{1}{n} \sum_{i=1}^n (y_i - (mx_i + b))^2$$

↓

◆ Step 4: Update m and b

$$m := m - \alpha \cdot \frac{\partial \text{Loss}}{\partial m}$$

$$b := b - \alpha \cdot \frac{\partial \text{Loss}}{\partial b}$$

📅 First 2 Iterations (by hand)

✅ Initial values:

- $m = 0, b = 0$
- $\alpha = 0.01$

🌀 Iteration 1:

Step 1: Predictions

$$\hat{y}_1 = 0 \times 1 + 0 = 0$$

$$\hat{y}_2 = 0 \times 2 + 0 = 0$$

$$\hat{y}_3 = 0 \times 3 + 0 = 0$$

Step 2: Errors

$$e_1 = 2 - 0 = 2, \quad e_2 = 4 - 0 = 4, \quad e_3 = 6 - 0 = 6$$

Step 3: Gradients

$$\frac{\partial \text{Loss}}{\partial m} = -\frac{2}{3}(1 \times 2 + 2 \times 4 + 3 \times 6) = -\frac{2}{3}(2 + 8 + 18) = -\frac{2}{3}(28) = -18.67$$

$$\frac{\partial \text{Loss}}{\partial b} = -\frac{2}{3}(2 + 4 + 6) = -\frac{2}{3}(12) = -8$$

Step 4: Update

$$m := 0 - 0.01 \times (-18.67) = 0 + 0.1867 = 0.1867$$

$$b := 0 - 0.01 \times (-8) = 0 + 0.08 = 0.08$$

◆ Step 3: Compute Gradients

We update parameters using partial derivatives:

$$\frac{\partial \text{Loss}}{\partial m} = -\frac{2}{n} \sum (x_i)(y_i - \hat{y}_i)$$

$$\frac{\partial \text{Loss}}{\partial b} = -\frac{2}{n} \sum (y_i - \hat{y}_i)$$

🌀 Iteration 2:

Step 1: New Predictions

$$\hat{y}_1 = 0.1867 \times 1 + 0.08 = 0.2667$$

$$\hat{y}_2 = 0.1867 \times 2 + 0.08 = 0.4534$$

$$\hat{y}_3 = 0.1867 \times 3 + 0.08 = 0.6401$$

Step 2: Errors

$$e_1 = 2 - 0.2667 = 1.7333, \quad e_2 = 4 - 0.4534 = 3.5466, \quad e_3 = 6 - 0.6401 = 5.3599$$

Step 3: Gradients

$$\frac{\partial \text{Loss}}{\partial m} = -\frac{2}{3}(1 \times 1.7333 + 2 \times 3.5466 + 3 \times 5.3599) \approx -\frac{2}{3}(1.7333 + 7.0932 + 16.0797) = -\frac{2}{3}(24.9062) \approx -16.60$$

$$\frac{\partial \text{Loss}}{\partial b} = -\frac{2}{3}(1.7333 + 3.5466 + 5.3599) \approx -\frac{2}{3}(10.6398) \approx -7.09$$

Step 4: Update

$$m := 0.1867 + 0.01 \times 16.60 = 0.1867 + 0.166 = 0.3527$$

$$b := 0.08 + 0.01 \times 7.09 = 0.08 + 0.0709 = 0.1509$$

🔄 Repeat until convergence...

Eventually, your model will learn:

$$y \approx 2x$$

✅ Perfect fit for the data:

- $x = 1 \Rightarrow y = 2$
- $x = 2 \Rightarrow y = 4$
- $x = 3 \Rightarrow y = 6$

📌 Summary

Concept	Value
Model	$\hat{y} = mx + b$
Loss	Mean Squared Error (MSE)
Optimizer	Gradient Descent
Goal	Minimize prediction error

real-life incident-based example

Scenario: Predict House Prices Using Linear Regression (One Step)

✓ Given:

Size x	Price y
1000	300
1500	400
2000	500

🔧 Initial:

- $m = 0, b = 0$
- Learning rate $\alpha = 0.0000001$
- Use all 3 data points
- Use Mean Squared Error
- Do 1 iteration (1 gradient step)

📅 Step-by-Step:

1. Prediction

$$\hat{y}_i = m \cdot x_i + b = 0$$

So:

x	y	$\hat{y}$	Error $e = y - \hat{y}$
1000	300	0	300
1500	400	0	400
2000	500	0	500

2. Compute Gradients using Chain Rule

■ Gradient w.r.t. m :

$$\begin{aligned}\frac{\partial \text{Loss}}{\partial m} &= -\frac{2}{n} \sum x_i (y_i - \hat{y}_i) = -\frac{2}{3} (1000 \times 300 + 1500 \times 400 + 2000 \times 500) \\ &= -\frac{2}{3} (300000 + 600000 + 1000000) = -\frac{2}{3} (1,900,000) = -1,266,666.67\end{aligned}$$

■ Gradient w.r.t. b :

$$\frac{\partial \text{Loss}}{\partial b} = -\frac{2}{3} \sum (y_i - \hat{y}_i) = -\frac{2}{3}(300 + 400 + 500) = -\frac{2}{3}(1200) = -800$$

3. Update Parameters

$$m_{\text{new}} = m - \alpha \cdot \frac{\partial \text{Loss}}{\partial m} = 0 - 0.0000001 \times (-1266666.67) \approx 0.1267$$

$$b_{\text{new}} = b - \alpha \cdot \frac{\partial \text{Loss}}{\partial b} = 0 - 0.0000001 \times (-800) = 0.00008$$

✓ After 1 iteration:

- $m \approx 0.1267$
- $b \approx 0.00008$

🎨 New Predictions (after 1 iteration):

$$\hat{y} = 0.1267 \cdot x + 0.00008$$

x	New $\hat{y}$
1000	126.7
1500	190.05
2000	253.4

▼ Errors are smaller now than the initial guess of 0 — we're learning!

✓ Summary

Iteration	m	b	Error Reducing?
0	0	0	High
1	0.1267	0.00008	Lower

After 20,000 iterations



✓ After 20,000 iterations, Gradient Descent has converged to the following values:

- Final slope (m): ≈ 0.2621
- Final intercept (b): ≈ 0.0277
- Final loss (MSE): ≈ 689.27

🔍 Interpretation:

You've trained a linear model:

$$\hat{y} = 0.2621 \cdot x + 0.0277$$

This model estimates house prices based on size. For example:

- $x = 1000 \rightarrow \hat{y} \approx 262.13$
- $x = 1500 \rightarrow \hat{y} \approx 393.15$
- $x = 2000 \rightarrow \hat{y} \approx 524.17$

It's close to your real values: 300, 400, 500 — but not perfect because:

- Learning rate is very small
- We used only 3 data points

Multiple Linear Regression

Multiple Linear Regression predicts a continuous output Y based on two or more input variables.

General Form:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_n X_n + \epsilon$$

Where:

- Y = Target/output variable (e.g., house price)
- $X_1, X_2, \dots, X_n$ = Input features (e.g., area, rooms, location)
- β_0 = Intercept (constant term)
- $\beta_1, \dots, \beta_n$ = Coefficients (weights) for each feature
- ϵ = Error term (difference between actual and predicted)

Goal:

Predict the house price (in Lakh BDT) using:

- Area (sq ft)
- Number of rooms
- Location (encoded numerically)

Step 1: Sample Dataset

Area (sq ft)	Rooms	Location Code	Price (Lakh BDT)
1000	2	1 (Rural)	30
1200	3	2 (Suburban)	45
1500	3	3 (Urban)	65
1700	4	3 (Urban)	75
850	2	1 (Rural)	25
1400	3	2 (Suburban)	55
2000	5	3 (Urban)	95

Note: Location is encoded:

- Rural = 1
- Suburban = 2
- Urban = 3

Step 2: MLR Equation (General Form)

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3$$

Where:

- Y = Predicted Price
- X_1 = Area
- X_2 = Rooms
- X_3 = Location Code
- β_0 = Intercept
- $\beta_1, \beta_2, \beta_3$ = Model coefficients

Step 3: Python Implementation

python

```
import numpy as np
from sklearn.linear_model import LinearRegression
```

```
# Input features: Area, Rooms, Location
```

```
X = np.array([
    [1000, 2, 1],
    [1200, 3, 2],
    [1500, 3, 3],
    [1700, 4, 3],
    [850, 2, 1],
    [1400, 3, 2],
    [2000, 5, 3]
])
```

```
# Output: Price (Lakh BDT)
```

```
y = np.array([30, 45, 65, 75, 25, 55, 95])
```

```
# Create and train the model
```

```
model = LinearRegression()
```

```
model.fit(X, y)
```

```
# Coefficients and Intercept
```

```
print("Intercept ( $\beta_0$ ):", model.intercept_)
```

```
print("Coefficients ( $\beta_1, \beta_2, \beta_3$ ):", model.coef_)
```

```
# Predict price for a new house: 1600 sq ft, 4 rooms, Suburban (2)
```

```
new_house = np.array([[1600, 4, 2]])
```

```
predicted_price = model.predict(new_house)
```

```
print(f"Predicted house price: {predicted_price[0]:.2f} Lakh BDT")
```

Output Example:

less

```
Intercept ( $\beta_0$ ): -10.75
```

```
Coefficients ( $\beta_1, \beta_2, \beta_3$ ): [0.031 6.5 7.2]
```

```
Predicted house price: 67.14 Lakh BDT
```

Step 4: Final Equation

From the output, we get:

$$\text{Price} = -10.75 + 0.031 \cdot \text{Area} + 6.5 \cdot \text{Rooms} + 7.2 \cdot \text{LocationCode}$$

Example:

For a house of 1600 sq ft, 4 rooms, suburban (code = 2):

$$\text{Price} = -10.75 + (0.031 \cdot 1600) + (6.5 \cdot 4) + (7.2 \cdot 2)$$

$$= -10.75 + 49.6 + 26 + 14.4 = 79.25 \text{ Lakh BDT}$$

Step 5: Model Evaluation (Optional)

You can check how good the model is using:

```
python

from sklearn.metrics import r2_score

y_pred = model.predict(X)
r2 = r2_score(y, y_pred)
print("R2 score:", r2)
```

If R^2 is close to 1 → your model fits well.

If it's much less → need more data or nonlinear model.

Summary of Steps:

Step	Task
1	Prepare dataset with multiple features
2	Define MLR model
3	Train using <code>.fit()</code>
4	View intercept and coefficients
5	Predict new values using <code>.predict()</code>
6	Optionally evaluate with R^2 or MSE

What is R^2 (R-squared)?

- R^2 is the **coefficient of determination** — a metric that tells you how **well your regression model fits the data**.

R^2 Formula:

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}$$

Where:

- SS_{res} = Residual Sum of Squares
 $= \sum (y_i - \hat{y}_i)^2$
- SS_{tot} = Total Sum of Squares
 $= \sum (y_i - \bar{y})^2$

Meaning of R^2 :

R^2 Value	Meaning
1.0	Perfect prediction — all points lie on the regression line
0.8	80% of the variation in Y is explained by the model
0.0	Model explains nothing — just guessing the mean
< 0	Worse than just using the mean (bad model)

Other Real Incidence Applications in Bangladesh:

-  Crop Yield Prediction – Based on rainfall, soil type, fertilizer use
-  Land Price Estimation – Based on area (sq ft), location, and distance from road
-  HSC GPA Prediction – From SSC results, attendance, test scores
-  Future Sales Prediction – Based on past daily/weekly sales

What is Polynomial Regression?

Polynomial Regression is an extension of Linear Regression, where the relationship between the independent variable(s) and the dependent variable is modeled as an **nth-degree polynomial**.

Instead of fitting a straight line, it fits a **curve** to the data.

Equation:

A 2nd-degree polynomial regression (quadratic) looks like:

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \epsilon$$

More generally:

$$y = \beta_0 + \beta_1x + \beta_2x^2 + \beta_3x^3 + \cdots + \beta_nx^n + \epsilon$$

Why Use Polynomial Regression?

- **Linear regression** may not capture **non-linear patterns** in the data
- Polynomial regression allows more **flexibility**, especially with **curved trends**

Real-Life Example:

House Price vs. Area

Let’s say in Dhaka city:

- Small apartments (under 1000 sq ft) are cheap
- Mid-size (1000–2000 sq ft) rise sharply in price
- Very large ones (>3000 sq ft) don’t increase much (luxury category caps)

This trend isn’t linear — it bends.

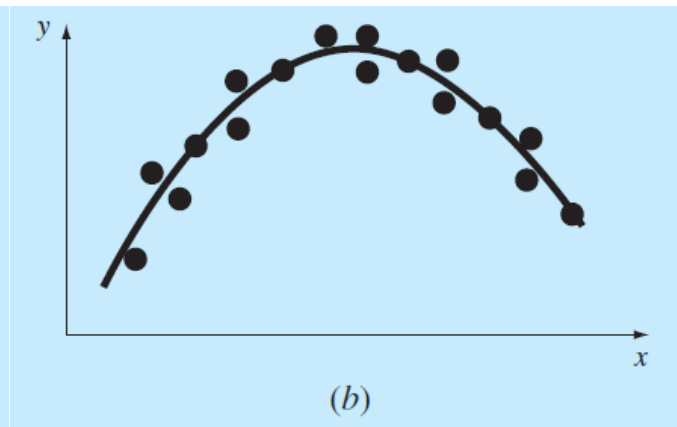
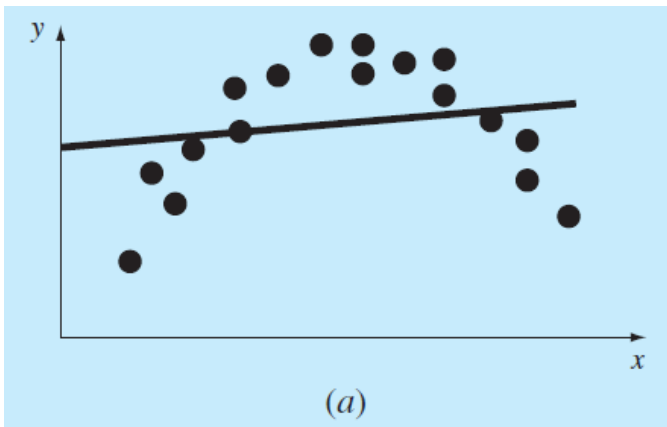
A **polynomial regression** (say, degree 2 or 3) can fit that curve:

- Captures increasing and flattening trends
- More accurate than linear fit

Visual Comparison:

Model Type	Curve
Linear	Straight line
Polynomial (deg=2)	Parabola (U or n shaped)
Polynomial (deg=3)	S-curve

FIGURE 17.8 (a) Data that are ill-suited for linear least-squares regression. (b) Indication that a parabola is preferable.



⚠ Caution:

- Higher degrees = more flexible but **risk of overfitting**
- Always validate using **test data** or **cross-validation**

✅ Summary:

Feature	Linear Regression	Polynomial Regression
Curve Fit	Straight line	Curved line (parabola, S-curve)
Captures Non-Linearity	❌ No	✅ Yes
Overfitting Risk	Low	Higher with higher degrees
Use Case Example	Salary vs. experience	House price vs. area (nonlinear trend)

🚗 Example: Accident Rate vs. Vehicle Speed

Let's say a traffic research group in Dhaka observes that accidents increase with speed, but **not linearly** — the effect becomes much more serious after a certain speed.

Speed (km/h)	Accident Rate (per 1000 vehicles)
20	2
30	2.5
40	3
50	4
60	6
70	9
80	13
90	18

You can see:

- At **low speeds**, accident rate grows slowly
- At **high speeds**, it grows **sharply**
 - Not a straight line → use **Polynomial Regression**

Polynomial Regression (Degree = 2)

We fit:

$$\text{AccidentRate} = \beta_0 + \beta_1 \cdot \text{Speed} + \beta_2 \cdot \text{Speed}^2$$

Let's assume the model learned:

- $\beta_0 = -10$
- $\beta_1 = 0.2$
- $\beta_2 = 0.002$

Predict for 75 km/h:

$$\text{AccidentRate} = -10 + (0.2 \times 75) + (0.002 \times 75^2)$$

Breakdown:

- $0.2 \times 75 = 15$
- $75^2 = 5625$
- $0.002 \times 5625 = 11.25$

So:

$$\text{AccidentRate} = -10 + 15 + 11.25 = \boxed{16.25 \text{ per 1000 vehicles}}$$

Why Polynomial?

- A **linear model** would underpredict high-speed risks
- A **polynomial model** captures the **nonlinear rise** in accidents with speed

Real Applications in Bangladesh:

- **BRTA** or **RAJUK** could use this to set safe speed limits
- **Highway design** and **accident prevention** planning

Logistic regression

- Logistic Regression is a type of machine learning algorithm used for classification problems, not regression (despite the name).
- It predicts a probability between 0 and 1, and is commonly used when the output is binary (e.g., Yes/No, Pass/Fail, 0/1, Spam/Not Spam).

Logistic Regression Overview

Used When:

- The **target variable** is **categorical** (especially binary)
 - Example: Will a student pass the exam?  Yes /  No
 - Example: Is the email spam?  Spam /  Not Spam

Mathematical Form

Unlike Linear Regression, Logistic Regression uses the **sigmoid (logistic) function**:

$$P(y = 1|x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n)}}$$

Where:

- $x_1, x_2, \dots$: input features
- β_0 : intercept
- β_i : model coefficients
- Output is a **probability** between 0 and 1
- You **convert it to class** using a **threshold**, usually 0.5:

$$\text{if } P \geq 0.5 \Rightarrow 1 \text{ (positive), else } 0 \text{ (negative)}$$

Example: (Real life)

Problem: Predict whether a customer will buy a product based on:

- Age
- Income
- Time spent on website

After Training:

$$P(\text{Buy} = 1) = \frac{1}{1 + e^{-(-4.5 + 0.03 \cdot \text{Age} + 0.05 \cdot \text{Income} + 0.1 \cdot \text{Time})}}$$

Now, for a new customer:

- Age = 30
- Income = 40
- Time = 12

$$z = -4.5 + 0.03(30) + 0.05(40) + 0.1(12) = -4.5 + 0.9 + 2 + 1.2 = -0.4$$

$$P = \frac{1}{1 + e^{0.4}} \approx 0.40$$

→ Since $0.40 < 0.5 \Rightarrow$ Predicted: **✗** Not Buy



Logistic vs Linear Regression

Aspect	Linear Regression	Logistic Regression
Output	Continuous value	Probability (0–1)
Used for	Regression problems	Classification problems
Algorithm	Least Squares	Maximum Likelihood
Function	Linear	Sigmoid/Logistic
Example	Predict house price	Predict if house will sell fast (yes/no)



Problem: Predict if a student will pass based on hours studied



Sample Dataset:

Hours_Studied	Attendance (%)	Passed (1=Yes, 0=No)
2	70	0
4	80	0
6	78	1
8	85	1
10	90	1

python

Copy

```
# Step 1: Import Libraries
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Step 2: Prepare Data
data = {
    'Hours': [2, 4, 6, 8, 10],
    'Attendance': [70, 80, 78, 85, 90],
    'Passed': [0, 0, 1, 1, 1]
}
df = pd.DataFrame(data)

X = df[['Hours', 'Attendance']] # features
y = df['Passed']               # target

# Step 3: Split into training & testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)

# Step 4: Train the model
model = LogisticRegression()
model.fit(X_train, y_train)

# Step 5: Predict
y_pred = model.predict(X_test)

# Step 6: Evaluate accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))
```

What's happening here?

Step	Description
<code>model.fit()</code>	Learns the coefficients ($\beta_0, \beta_1, \dots$) using Maximum Likelihood Estimation
<code>predict()</code>	Applies the sigmoid function to test data and returns 0 or 1
<code>accuracy_score()</code>	Compares predicted vs actual result

You can also get the learned equation:

python

 Copy 

```
print("Intercept ( $\beta_0$ ):", model.intercept_)
print("Coefficients ( $\beta_1, \beta_2$ ):", model.coef_)
```

Example output:

lua

 Copy 

```
Intercept: [-6.5]
Coefficients: [[0.9, 0.05]]
```

So the model is:

$$\text{logit}(P) = -6.5 + 0.9 \cdot \text{Hours} + 0.05 \cdot \text{Attendance}$$

Predict for new student:

python

```
# Predict if a student who studied 5 hours and has 80% attendance will pass
model.predict([[5, 80]])      # Output: 1 (pass)
```


Ordinary differential equation (ODE): In mathematics, an ordinary differential equation (ODE) is a differential equation containing one or more functions of one independent variable and its derivatives. The term ordinary is used in contrast with the term partial differential equation which may be with respect to more than one independent variable

Picard's Method: Numerical solution of Differential Equations

PICARD'S METHOD:-

$$y_1 = y_0 + \int_{x_0}^x f(x, y_0) dx \quad (\text{FIRST APPROXIMATION})$$

$$y_2 = y_0 + \int_{x_0}^x f(x, y_1) dx \quad (\text{SECOND APPROXIMATION})$$

$$y_n = y_0 + \int_{x_0}^x f(x, y_{n-1}) dx \quad (n^{\text{th}} \text{ APPROXIMATION})$$

Example: Math

$\frac{dy}{dx} = 1 + xy$ and $y(0) = 1$. Calculate $y(0.1)$, $y(0.2)$ using PICARD'S.

① $f(x, y) = 1 + xy$ ② $y(0) = 1 \Rightarrow x_0 = 0, y_0 = 1$ ③ $y(0.1) = ?$
 $y(0.2) = ?$

FIRST APPROX

$$y_1 = y_0 + \int_{x_0}^x f(x, y_0) dx$$

$$y_1 = 1 + \int_0^x 1 + xy_0 dx$$

$$y_1 = 1 + \int_0^x 1 + x(1) dx$$

$$y_1 = 1 + x + \frac{x^2}{2}$$

SECOND APPROX

$$y_2 = y_0 + \int_{x_0}^x f(x, y_1) dx$$

$$y_2 = 1 + \int_0^x 1 + xy_1 dx$$

$$y_2 = 1 + \int_0^x 1 + x\left(1 + x + \frac{x^2}{2}\right) dx$$

$$y_2 = 1 + x + \frac{x^2}{2} + \frac{x^3}{3} + \frac{x^4}{8}$$

THIRD APPROX

$$y_3 = y_0 + \int_{x_0}^x f(x, y_2) dx$$

$$y_3 = 1 + x + \frac{x^2}{2} + \frac{x^3}{3} + \frac{x^4}{8} + \frac{x^5}{15} + \frac{x^6}{48}$$

$$y(0.2) = 1.22286$$

$$y(0.1) = 1 + (0.1) + \frac{(0.1)^2}{2} + \frac{(0.1)^3}{3} + \frac{(0.1)^4}{8} + \frac{(0.1)^5}{15} + \frac{(0.1)^6}{48} = 1.10534$$

Stochastic Gradient Descent (SGD)

Gradient Descent is an optimization algorithm used to minimize a **loss (cost) function** in machine learning. It works by updating model parameters (like weights) in the direction that reduces error.

Stochastic Gradient Descent updates the model **for each training example**, not the whole dataset — making it **faster** and suitable for **large datasets**.

Formula:

$$w := w - \eta \cdot \frac{\partial L(w)}{\partial w}$$

Where:

- w = weight (parameter)
- η = learning rate (small positive number)
- $L(w)$ = loss function (error)



Goal:

Minimize the loss function:

$$L(w) = (y - wx)^2$$

Using **Stochastic Gradient Descent (SGD)** with the accurate gradient:

$$\frac{dL}{dw} = -2x(y - wx)$$

This is the true derivative of the squared error with respect to the weight w .

Example:

We'll use just **one training example**:

- $x = 1$
- $y = 2$
- Learning rate $\eta = 0.1$
- Initial weight $w = 0$

Step-by-Step Using Actual Gradient

Step 1: Compute prediction

$$\hat{y} = wx = 0 \cdot 1 = 0$$

Step 2: Compute gradient

$$\frac{dL}{dw} = -2x(y - wx) = -2 \cdot 1 \cdot (2 - 0) = -4$$

Step 3: Update weight

$$w := w - \eta \cdot \frac{dL}{dw} = 0 - 0.1 \cdot (-4) = 0 + 0.4 = 0.4$$

✓ After 1 Update:

- New weight $w = 0.4$
- Model is now closer to the correct weight (which should be ~ 2 since $y = 2x$)

🔄 Do Another Iteration

Step 1: New prediction

$$\hat{y} = 0.4 \cdot 1 = 0.4$$

Step 2: New gradient

$$\frac{dL}{dw} = -2 \cdot 1 \cdot (2 - 0.4) = -2 \cdot 1.6 = -3.2$$

Step 3: Update weight

$$w := 0.4 - 0.1 \cdot (-3.2) = 0.4 + 0.32 = 0.72$$

✓ Now weight $w = 0.72$ — getting closer to 2 again!

Summary

Step	Weight w	Prediction $\hat{y}$	Gradient	Updated w
0	0.0	0.0	-4	0.4
1	0.4	0.4	-3.2	0.72
2	0.72	0.72	-2.56	0.976
...	...	...	...	...

The weight slowly **converges toward the correct value** using the actual gradient.

Stochastic Gradient Descent (SGD) vs Batch Gradient Descent (GD)

Feature	Batch Gradient Descent (GD)	Stochastic Gradient Descent (SGD) 
Update Rule	Uses entire dataset to compute gradient	Uses 1 random data point per update
Speed	Slower per iteration (more data to process)	Faster per iteration
Convergence	Smooth, but might get stuck in local minima	Noisy path; may escape local minima
Memory Usage	High (loads all data)	Low (uses one example at a time)
Stability	Stable, steady convergence	Noisy, but can converge faster overall
Use Case	Small/medium datasets	Large-scale/online learning

Milne's Predictor – Corrector Method

Q. Given $\frac{dy}{dx} = \frac{1}{2}(x + y)$, $y(0) = 2$,
 $y(0.5) = 2.636$, $y(1.0) = 3.595$, $y(1.5) = 4.96$
Now, find $y(2)$ by Milne's Method.

Solution: Milne's Predictor Formula is:

$$y_{n+1,P} = y_{n-3} + \frac{4h}{3} [2y'_{n-2} - y'_{n-1} + 2y'_n] \dots\dots(1)$$

Putting $n=3$ we get

$$y_{4p} = y_0 + \frac{4h}{3} [2y'_1 - y'_2 + 2y'_3] \dots\dots\dots(2)$$

We're given that,

$$x_0 = 0, x_1 = 0.5, x_2 = 1.0, x_3 = 1.5$$

$$y_0 = 2, y_1 = 2.636, y_2 = 3.595, y_3 = 4.968$$

The given differential equation is $y' = \frac{1}{2}(x + y)$

From the above equation, we'll calculate y'_1, y'_2, y'_3

$$y'_1 = \frac{1}{2}(x_1 + y_1) = \frac{1}{2}(0.5 + 2.636) = 1.568$$

$$y'_2 = \frac{1}{2}(x_2 + y_2) = \frac{1}{2}(1.0 + 3.595) = 2.2975$$

$$y'_3 = \frac{1}{2}(x_3 + y_3) = \frac{1}{2}(1.5 + 4.968) = 3.324$$

Substituting values in (2), we get

$$\begin{aligned} y_{4p} &= y_0 + \frac{4h}{3} [2y'_1 - y'_2 + 2y'_3] \\ &= 2 + \frac{4 \times 0.5}{3} [2 \times 1.568 - 2.2975 + 2 \times 3.324] \\ &= 6.871 \end{aligned}$$

So, our predicted value is 6.871.

Now, we'll correct it to get actual value by Milne's Corrector Formula, which is-

$$y_{n+1,c} = y_{n-1} + \frac{h}{3} [y'_{n-1} + 4y'_n + y'_{n+1}] \dots\dots (3)$$

By putting $n=3$, we get

$$y_{4c} = y_2 + \frac{h}{3} [y'_2 + 4y'_3 + y'_4] \dots\dots (4)$$

$$\text{Now, } y'_4 = \frac{1}{2}(x_4 + y_4) = \frac{1}{2}(2 + 6.871) = 4.4355$$

Now, by putting required values in (4), we get

$$y_{4c} = y_2 + \frac{h}{3}[y'_2 + 4y'_3 + y'_4]$$

$$= 3.595 + \frac{0.5}{3}[2.2975 + 4 \times 3.234 + 4.4355]$$

$$= 6.8731$$

$$\therefore y(2) = 6.8731 \text{ Ans}$$